ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY
UNIVERSITY FOR INDUSTRY

College of Engineering Department of Software Engineering

Course Title: Database Management System[SWEG2108]

Project Title*:* **YISAKAL SAVING AND CREDIT COOPERATIVE ORGANIZATION**

Section A, Group 5b(J)

Group Members	ID No.
Anael Mulugeta	ETS0176/17
Betelhem Hailu	ETS0298/17
Betelhem Sefiw	ETS0285/17
Bontu Bekele	ETS0361/17
Daniel Alemu	ETS0406/17
Dilayehu Dessalegn	ETS0445/17

Submitted to: Mr Yayneshet Medhin

Submission date: May 22,2026G.C

TABLE OF CONTENTS

Chapter one:

Chapter Two : Database Design

Chapter 3: Implementation

YISAKAL SAVING AND CREDIT COOPERATIVE ORGANIZATION.

(YISAKAL SACCO)

Hypothetical Organization Data Base System

Chapter 1: Problem Identification and Proposed Solution

1.1 Background of Yisakal SACCO

Yisakal Saving and Credit Cooperative Society (Yisakal SACCO) is a member-owned financial cooperative established in April 2026 in Addis Ababa. Its mission is to improve the socio-economic conditions of members by providing accessible savings, credit, and other financial services to low- and middle-income groups who are often underserved by commercial banks. The SACCO offers services such as voluntary and compulsory savings, individual and group loans, micro-business financing, and women-focused empowerment programs through its expanding branch network and core banking system.

As the number of members and financial transactions increases, managing data related to members, savings accounts, loans, guarantors, collateral, and repayments becomes more complex. To address this, Yisakal SACCO requires a robust database management system that ensures accurate record keeping, efficient transaction processing, data security, and reliable reporting. The proposed system supports daily SACCO operations while improving efficiency, consistency, accountability, and decision-making.

1.2 Problem Statement

Yisakal Saving and Credit Cooperative Society currently manages large volumes of data related to members, savings accounts, loans, repayments, and branch operations across its expanding network in Addis Ababa. As membership and transaction volumes increase, the existing data management approach has become difficult to maintain due to issues such as data redundancy, inconsistency, duplication, and delayed reporting.

Many operational activities, including member registration, loan processing, repayment tracking, and report generation, still depend on fragmented spreadsheets, semi-manual procedures, and partially integrated legacy systems. This limits operational efficiency and makes it difficult to access accurate real-time information, monitor overdue loans, and generate consolidated reports for management and regulatory purposes.
The project integrates both relational and Non-relational database technologies to overcome the limitations of traditional database systems. Therefore, there is a need to design and implement an integrated database system that combines a normalized MySQL relational database and also a NoSQL database such as MongoDB which we have done them analogously or are corresponding to each other.

1.3 What the organization does and its objectives

The SACCO Management System supports the management of savings and credit services for members in an efficient and secure way. It helps the organization handle financial and member information digitally.

- Mobilizing compulsory and voluntary savings from members.
- Providing different loan services such as business loans, education loans, emergency loans, and personal loans based on member eligibility and needs.
- Managing loan applications, approvals, disbursement, repayment, and monitoring overdue loans.
- Maintaining accurate records of members, accounts, transactions, guarantors, and collateral information.
- Improving financial service delivery through secure and efficient database management.
- Supporting financial transparency, accountability, and data consistency within the organization.
- Generating reports and maintaining audit records for administrative and financial monitoring.
- Providing financial awareness and capacity-building support for members, including women and youth groups.
- Supporting scalable and modern financial operations using both relational and NoSQL database technologies.

1.4 How services and loans are given to customers

Yisakal SACCO provides financial services through a structured and controlled loan process:

1. Membership and Saving Requirement:
   - A client becomes a member by registering and opening a savings account.
   - The member is required to save regularly and maintain a minimum balance before becoming eligible for loans.
2. Loan Application:
   - The member submits a loan request indicating loan type, purpose, amount, and repayment period.
   - Required documents such as identification, guarantors, and collateral details (if needed) are attached.
3. Appraisal and Approval :
   - Loan officers assess eligibility based on savings history, repayment capacity, and risk factors.
   - A loan committee reviews the application and approves, rejects, or adjusts the request.
4. Contract and Disbursement :
   - An agreement is signed outlining the loan amount, interest rate, repayment schedule, and terms.
   - The approved loan is disbursed to the member through their account or approved

payment method.

5. **Repayment and Monitoring**
   - Members repay the loan according to a scheduled plan (weekly or monthly).
   - Payments are recorded and monitored to track outstanding balances and detect overdue loans.
6. **Multiple Loan Handling**
   - Members may apply for additional or different loan types.
   - Approval depends on internal policies such as repayment performance, saving level, and existing loan balance.
   - The SACCO may allow multiple loans or require partial/full repayment of existing loans before issuing a new one.

1 . 5 Setbacks of the current database/information system

The current system faces several challenges including:

1 Fragmented and partly manual data handling

Some processes such as loan appraisal notes and member background information are still handled manually or stored in spreadsheets, which leads to incomplete integration and delays in accessing full records.

2 Limited support for semi-structured data

The relational database does not efficiently store detailed notes, scanned documents, and logs, so some important information is kept outside the main system.

3 Reporting and decision-making limitations

Generating detailed reports such as risk analysis and portfolio performance requires manual collection of data from different sources, which slows decision-making.

4 Data quality and duplication issues

Inconsistent data entry can result in duplicate or inaccurate records, affecting the reliability of member and loan information.

5 Scalability and integration challenges

As the system grows, handling large volumes of transactions and integrating new services becomes more difficult without a more scalable design.

1.6 General and Specific Objectives of the project

1.6.1 General Objective

• To analyze the data processing activities of Yisakal SACCO and design and implement an integrated database system using MySQL and MongoDB to improve data management, operational efficiency, and reporting.

1.6.2 Specific Objectives

• To study the existing data processing activities of Yisakal SACCO, including member registration, savings, loans, repayments, and reporting, and identify major data management challenges.

• To design a normalized relational database schema for the core SACCO operations and implement it using MySQL and similary apply that in the Non-relational DB using MongoDB.

• To implement the designed MySQL and MongoDB databases for managing SACCO operations efficiently.

• To test and evaluate the implemented system in terms of functionality, data integrity, usability, and performance.

1.7 Scope

This project focuses on the main data processing activities of Yisakal SACCO using MySQL and MongoDB database systems. The system covers:

• Member management, including registration, profile management, and membership status tracking.

• Savings management, including account creation, deposits, withdrawals, and balance management.

• Loan management, including loan application, approval, disbursement, repayment tracking, guarantor and collateral management, and loan status monitoring.

• Transaction and audit management for maintaining financial records and system activities.

• Basic reporting on members, savings, loans, and repayment performance to support Management decisions.

The project does not include systems such as mobile banking, ATM integration, payroll management, or advanced regulatory reporting. It provides a scalable prototype that represents the main operational activities of Yisakal SACCO.

1.8 Methodology

1. Requirement gathering and problem identification to understand the existing data processing activities and challenges of Yisakal SACCO.
2. System analysis and design by identifying the main entities, relationships, and data flows, followed by designing the ER diagram and normalized database schema.
3. Database implementation using MySQL for the relational database and MongoDB for semi-structured data management.

1.9 Tools and Technologies used

1. MySQL – used for the core relational database, including tables, relationships, constraints, and transaction management.
2. MongoDB – used for managing semi-structured data such as logs, audit records, and document-based information.
3. MySQL Workbench – used for database design, query execution, and relational database management.
4. MongoDB Compass / Mongo Shell – used for managing MongoDB collections and queries.
5. Lucidchart – used for designing ER diagrams and system diagrams.
6. GitHub and Google Docs – used for project documentation and collaboration.
7. JavaScript and JSON – used for scripting and handling MongoDB document structures.

1.10 SURVEY QUESTIONS ASKED ABOUT YISAKAL SACCO

The following survey questions and answers are hypothetical and prepared for academic purposes only. They are designed to represent possible requirements and responses that may be obtained from stakeholders of Yisakal SACCO during system analysis and database design. The responses do not reflect actual organizational data but are used to support the development of a conceptual database system.

1. What services does Yisakal SACCO provide to its members?

Yisakal SACCO provides a variety of financial and cooperative services to its members. It offers voluntary savings accounts that allow flexible deposits and withdrawals, and compulsory savings schemes that help members build financial discipline and long-term savings habits. The SACCO also provides individual loans for personal needs and business development, as well as group loans that support collective borrowing for organized members. In addition, it offers micro-business financing to support small and emerging entrepreneurs in expanding their businesses. Furthermore, Yisakal SACCO runs women empowerment programs aimed at improving financial inclusion, encouraging entrepreneurship, and supporting sustainable economic development among its members.

2. What are the main services, goals, and reporting needs of Yisakal SACCO Management Office?

At the branches of Yisakal SACCO, the management office is responsible for overseeing savings, loans, and overall financial operations. It requires regular reports on member growth, loan performance, savings balances, branch performance, and financial summaries to support planning and decision-making.

3. What are the key operational activities and data management requirements of Yisakal SACCO Branch Administration?

At the branches of Yisakal SACCO, the branch administration is responsible for handling daily operations such as member registration, savings transactions, loan processing, repayments, and record management. It also requires accurate and timely data handling to support reporting, decision-making, and coordination with the central management office.

4. What are the loan processing, approval, and repayment management requirements of the Yisakal SACCO Loan Department?

At the branches of Yisakal SACCO, the loan department is responsible for evaluating loan applications, approving eligible members, and monitoring repayment schedules. It also manages loan records, interest calculations, guarantor and collateral information, and ensures proper tracking of outstanding balances and overdue payments.

5. What are the financial management and accounting requirements of the Yisakal SACCO

system?

At the branches of Yisakal SACCO, the Finance & Accounting Department manages all financial transactions including savings, loans, repayments, and fees. It also prepares financial reports and ensures accurate record keeping, interest calculation, and account recognition for decision-making and auditing.

6. What are the service needs and experience expectations of Yisakal SACCO members?

At the branches of Yisakal SACCO, members mainly require efficient access to savings and loan services, including deposits, withdrawals, and loan applications. They also expect fast processing of transactions, clear account information, and timely updates on their balances, loan status, and repayments.

7. What are the system, security, and technical requirements of the Yisakal SACCO IT/System Department?

At the branches of Yisakal SACCO, the IT/System Department requires a secure and reliable database system that supports user access control, data backup, and recovery. It also needs real-time data processing, system stability, and proper integration between branches to ensure accurate and consistent information across all operations.

8. What are the challenges and limitations of the current data handling system at Yisakal SACCO?

At the branches of Yisakal SACCO, the current data handling system faces several challenges. These include data redundancy and inconsistency due to manual record keeping, which often leads to duplicated or conflicting information. Report generation is slow and time-consuming, affecting timely decision-making. In addition, tracking loans and repayments across different branches is difficult and error-prone. The system also lacks real-time data access and has weak backup and security mechanisms, increasing the risk of data loss.

9. What solutions or improvements are expected from the future database system?

At the branches of Yisakal SACCO, the future database system is expected to eliminate data redundancy by using a centralized database. It should provide real-time access to member, loan, and savings information across all branches. The system should automate report generation, improve loan and transaction tracking, and enhance data security through backup and user access control. It is also expected to improve efficiency, accuracy, and overall decision-making in SACCO operations.

10. How can the benefit of replacing the existing system with a database system be measured?

At the branches of Yisakal SACCO, the benefits can be measured by improved speed in data processing and reporting, reduced errors and duplication, faster loan and transaction handling, and better accuracy of records. It can also be seen in improved efficiency, stronger data security, and more reliable decision-making.

11. What collections of related tables/records are managed in the Yisakal SACCO system?

At the branches of Yisakal SACCO, the system manages related tables for members, savings, loans, transactions, employees, branches, and fees. These records are regularly updated and used

for daily operations and reporting.

Chapter Two : Database Design

2.1 Input Design for Yisakal SACCO Management System (Part of Annex)

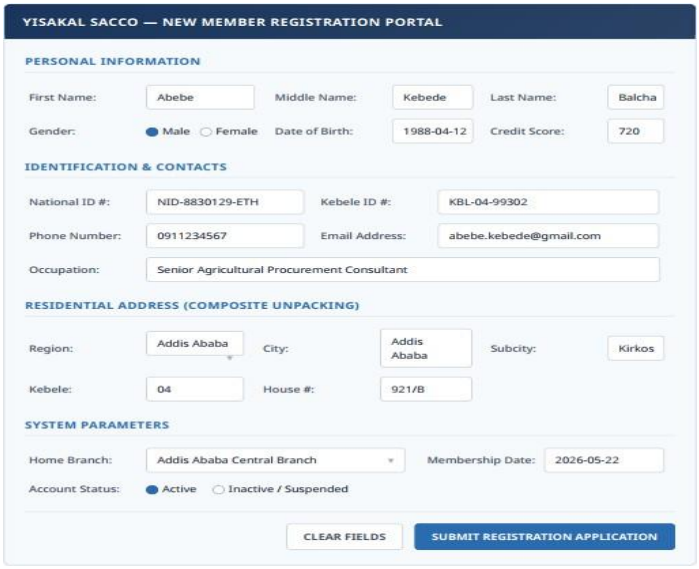
This chapter explains the screens and reports used in the Yisakal SACCO system, such as forms, menus, and dashboards.

Its main goal is to make sure that everything users see and interact with on the screen directly matches the database structure behind the system. This includes input fields, selections, tables, and reports.

In short, it ensures that what users enter and view in the system is properly connected to the organized database so that data is stored correctly, consistently, and without errors.

This part explains how new member information is entered into the SACCO system using a registration form.

The form is used by staff to officially add a new client into the system. When the form is submitted, the system safely saves the member's details into the database in one complete step and makes sure there are no duplicates (for example, using a unique ID like a national identity number).

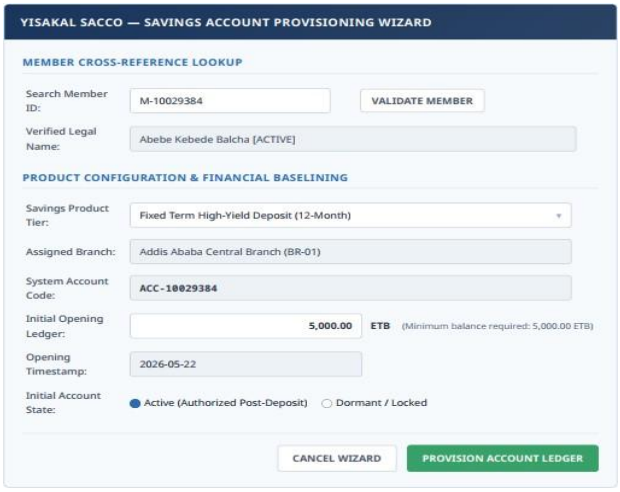


This section explains a form used to create a savings account for an existing member.

It is used when a member wants to open a savings account that earns interest. The form connects the account to the correct savings product type, links it to the right member, and records where and when the account was opened.

Form 2: Savings Account Guided Setup Tool (Provision Wizard)
This section explains a form used to create a savings account for an existing member.

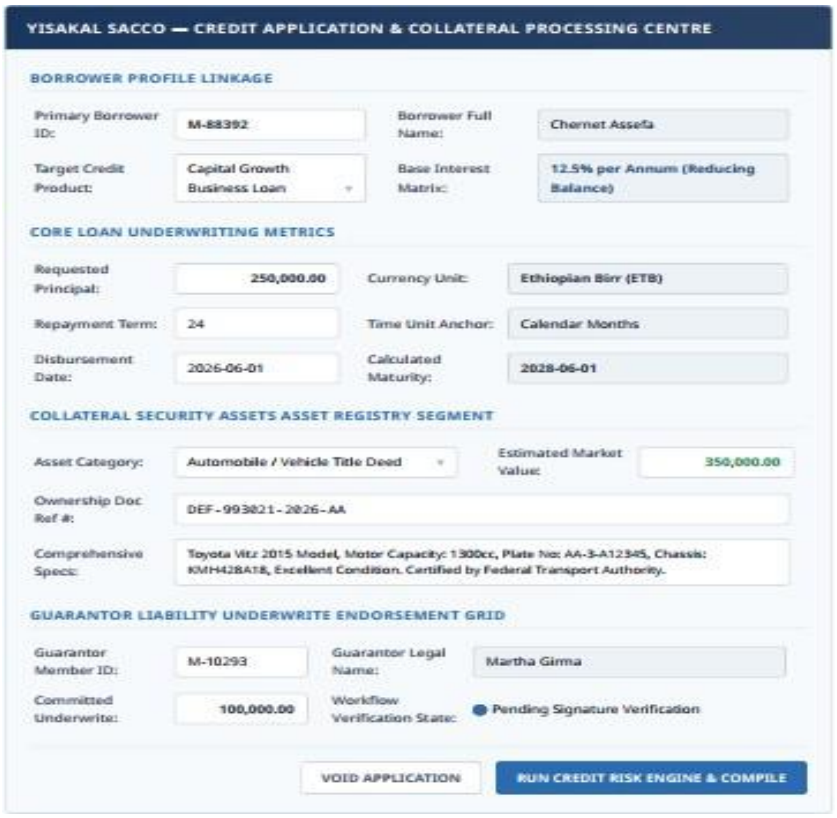
It is used when a member wants to open a savings account that earns interest. The form connects the account to the correct savings product type, links it to the right member, and records where and when the account was opened.



Form 3: Credit Application and Collateral Intake Dashboard

This form is used when a member applies for a loan and provides security for it.

It helps the system record the loan request, link it to any assets used as collateral, and include any guarantors who agree to take responsibility if the borrower fails to repay.


YISAKAL SACCO — CREDIT APPLICATION & COLLATERAL PROCESSING CENTRE
BORROWER PROFILE LINKAGE
Primary Borrower ID:
M-88392
Borrower Full Name:
Chernet Assefa
Target Credit Product:
Capital Growth Business Loan
Base Interest Matrix:
12.5% per Annum (Reducing Balance)
CORE LOAN UNDERWRITING METRICS
Requested Principal:
250,000.00
Currency Unit:
Ethiopian Birr (ETB)
Repayment Term:
24
Time Unit Anchor:
Calendar Months
Disbursement Date:
2026-06-01
Calculated Maturity:
2028-06-01
COLLATERAL SECURITY ASSETS ASSET REGISTRY SEGMENT
Asset Category:
Automobile / Vehicle Title Deed
Estimated Market Value:
350,000.00
Ownership Doc Ref #:
DEF-993821-2026-AA
Comprehensive Specs:
Toyota Vitz 2015 Model, Motor Capacity: 1300cc, Plate No: AA-3-A12345, Chassis: KMH428A18, Excellent Condition. Certified by Federal Transport Authority.
GUARANTOR LIABILITY UNDERWRITE ENDORSEMENT GRID
Guarantor Member ID:
M-10293
Guarantor Legal Name:
Martha Girma
Committed Underwrite:
100,000.00
Workflow Verification State:
Pending Signature Verification
VOID APPLICATION
RUN CREDIT RISK ENGINE & COMPILE

2.2 Conceptual Entity Relationship Diagram

This section describes the main entities of the Yisakal SACCO system, their attributes, and the relationships established between them. It also explains the cardinality constraints that define how entities interact within the database structure to support SACCO operations efficiently.

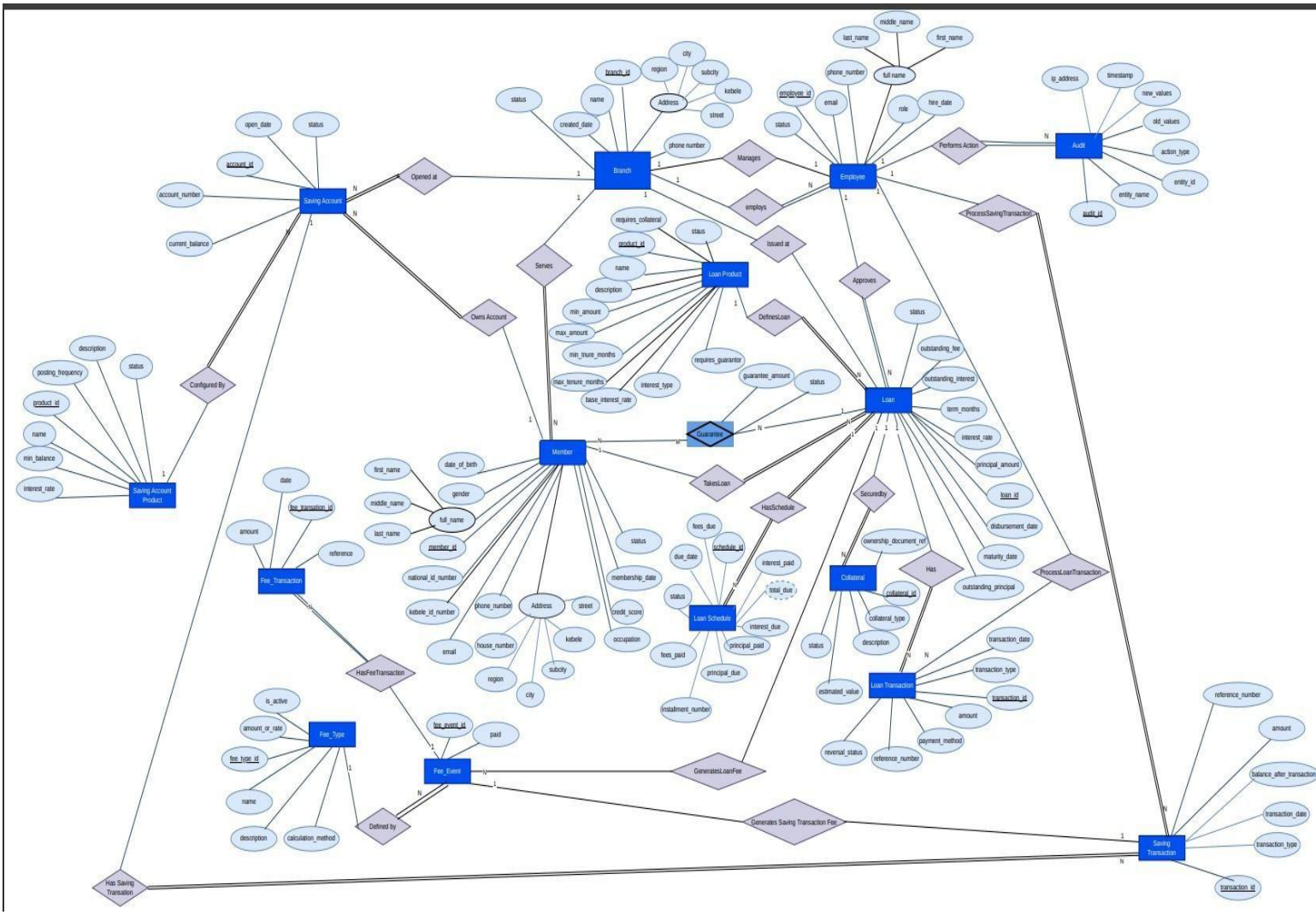


Figure 1: Conceptual Entity Relationship Diagram Of Yisakal SACCO

Definition of Important Terms

Entities: Real-world objects represented in the database, such as Member, Loan, and Branch.

Attributes: Characteristics or properties of an entity, such as name, ID, or phone number.

Relationships: Connections between entities in the database.

Cardinality: The number of relationships between entities, such as one-to-one or one-to-many.

2.2.1 Entity Identification and Attributes

This section identifies and defines each entity in the system, classifying them as **Strong**, **Weak**, or **Associative**.

1. **Branch (Strong Entity):** Represents a physical bank or microfinance branch office. It exists independently of any other entity.
   - Attributes
     - branch_id(PK) – Unique identifier of the branch.
     - name– Human-readable branch name (e.g., Downtown Branch).
     - region– Administrative region of the branch.
     - city– City where the branch is located.
     - subcity– Sub-city or district.
     - kebele– Local kebele name/number (can be left blank if not applicable).
     - street– Street name or description (can be left blank).
     - phone_number– Branch contact phone number.
     - manager_id(FK → Employee.employee_id) – Employee who manages this branch.
     - created_date– Date when the branch was opened/created.
     - status– Current status (Active, Closed).

2. **Employee (Strong Entity):** Represents the staff members working for the organization. It exists independently.

   - Attributes
     - employee_id(PK) – Unique identifier of the employee.
     - branch_id(FK → Branch.branch_id) – Branch where the employee is assigned.
     - full_name– Employee full name.
     - email– Employee email address.
     - phone_number– Employee phone number.
     - role– Role in the organization (Loan Officer, Teller, Manager, Admin).
     - hire_date– Date the employee was hired.
     - status– Employment status (Active, Suspended, Terminated).

3. **Member (Strong Entity):** Represents the customers or clients who open accounts and apply for loans.
   - Attributes
     - member_id(PK) – Unique identifier of the member.
     - branch_id(FK → Branch.branch_id) – Home or primary branch of this member.
     - full_name– Member's full legal name.
     - gender– Gender of the member.

- date_of_birth– Date of birth.
- national_id_number – National ID or government ID number.
- Kebele_id_number– Local kebele-issued ID number.
- email– Email address.
- phone_number– Primary phone number.
- region– Region where the member lives.
- city– City of residence.
- subcity– Sub-city or district.
- kebele– Kebele (can be blank if not applicable or unknown).
- street– Street or neighborhood description (can be blank).
- house_number– House number.
- occupation– Member's occupation.
- credit_score– Internal or external credit score assigned to the member.
- membership_date– Date when the member joined the SACCO.
- status– Membership status (Active, Inactive, Blacklisted).

4. **Loan Product (Strong Entity):** Represents the blueprint or configurations of financial loan types offered (e.g., Personal Loan, Business Loan).

- Attributes
  - product_id(PK) – Unique identifier of the loan product.
  - name– Product name (Personal Loan, Mortgage Loan, Business Loan, etc.).
  - description– Description of the loan product and its purpose.
  - min_amount– Minimum allowed principal amount.
  - max_amount– Maximum allowed principal amount.
  - min_tenure_months– Minimum loan term in months.
  - max_tenure_months– Maximum loan term in months.
  - base_interest_rate– Standard interest rate defined for this product.
  - interest_type– Interest calculation type (Flat, Declining Balance, Compound).
  - requires_guarantor– Flag indicating if guarantors are required.
  - requires_collateral– Flag indicating if collateral is required.
  - status – Product status (Active, Retired).

5. **Saving Account Product (Strong Entity):** Represents the configurations of available savings account types (e.g., Fixed Deposit, Student Savings).

- Attributes
  - product_id(PK) – Unique identifier of the saving product.
  - name– Product name (e.g., Compulsory Savings, Voluntary Savings).
  - description– Description of the product rules and usage.
  - min_balance– Minimum required balance (if applicable).
  - interest_rate– Standard interest rate for this product.
  - posting_frequency– How often interest is posted (monthly, quarterly,

yearly).
- status – Product status (Active, Retired).

6. **Saving Account (Strong Entity):** Represents an active, individual savings account owned by a member.

   attributes
   - account_id(PK) – Unique identifier of the saving account.
   - member_id(FK → Member.member_id) – Owner of the account.
   - account_number– Unique account number string used in operations.
   - branch_id(FK → Branch.branch_id) – Branch where the account is maintained.
   - product_type(FK → Saving Account Product.product_id) – The saving product this account belongs to.
   - current_balance– Current balance of the account.
   - open_date– Date when the account was opened.
   - status – Account status (Active, Dormant, Frozen, Closed).

7. **Loan (Strong Entity):** Represents a specific loan instance issued to a member. It tracks balances and terms.

- Attributes
  - loan_id(PK) – Unique identifier of the loan.
  - member_id(FK → Member.member_id) – Borrower who took the loan.
  - product_id(FK → Loan Product.product_id) – Loan product configuration used.
  - loan_officer_id(FK → Employee.employee_id) – Employee responsible for managing/approving this loan.
  - principal_amount– Amount originally borrowed.
  - interest_rate– Actual interest rate assigned to this loan (may differ from base).
  - term_months– Agreed loan term in months.
  - disbursement_date– Date when the loan was disbursed (funds given out).
  - maturity_date– Date when the loan is expected to be fully repaid.
  - outstanding_principal– Current unpaid principal balance.
  - outstanding_interest– Accrued but unpaid interest.
  - outstanding_fees– Accrued but unpaid fees related to the loan.
  - status– Loan status (Pending, Approved, Disbursed, Active, Closed, Defaulted).

8. **Guarantor:** (Associative Entity): it models members who guarantee loans for other members. It is an associative entity between loan and member.
- Attributes
  - loan_id(PK, FK → Loan.loan_id) – Guaranteed loan.

- member_id(PK, FK → Member.member_id) – Member acting as guarantor.
- guarantee_amount– Amount guaranteed by this member.
- status– Guarantor status (Active, Released).

(Composite primary key: loan_id, member_id.)

9. **Collateral (Strong Entity):** Represents assets pledged by a member to secure a loan.
- Attributes
  - collateral_id(PK) – Unique identifier of the collateral record.
  - loan_id(FK → Loan.loan_id) – Loan secured by this collateral.
  - collateral_type– Type of collateral (Real Estate, Vehicle, Equipment, Cash, etc.).
  - description– Text description of the asset.
  - estimated_value– Estimated monetary value of the asset.
  - ownership_document_ref– Reference to ownership document (ID, file, registration number).
  - status – Status of the collateral (Pledged, Released, Liquidated).

10. **Loan Schedule (Strong Entity):** Tracks the individual payment installments due for a specific loan.
- Attributes
  - schedule_id(PK) – Unique identifier of the schedule row.
  - loan_id(FK → Loan.loan_id) – Loan to which this installment belongs.
  - installment_number– Installment sequence (e.g., 1, 2, …, 36).
  - due_date– Due date for this installment.
  - principal_due– Principal amount due in this installment.
  - interest_due– Interest amount due in this installment.
  - fees_due– Fees amount due in this installment.
  - total_due– Total amount due (principal + interest + fees), derived attribute.
  - principal_paid– Principal amount actually paid for this installment so far.
  - interest_paid– Interest amount actually paid.
  - fees_paid– Fees amount actually paid.
  - status – Installment status (Pending, Partially Paid, Paid, Overdue).

11. **Loan Transaction (Strong Entity):** Represents individual financial actions (payments, disbursements) against a loan.

- Attributes
  - transaction_id(PK) – Unique identifier of the loan transaction.
  - loan_id(FK → Loan.loan_id) – Loan affected by this transaction.
  - transaction_type– Type of transaction (Disbursement, Repayment, Adjustment, Write-off).
  - amount– Transaction amount.
  - transaction_date– Date/time of the transaction.
  - payment_method– How the payment was made (Cash, Bank Transfer, Card, Mobile Money).
  - reference_number– External receipt ID or tracking number.

- employee_id(FK → Employee.employee_id) – Staff member who processed the transaction.
- reversal_status– Reversal state (None, Reversed).

12. **Saving Transaction (Strong Entity):** Tracks individual financial actions (deposits, withdrawals) against a savings account.

- Attributes
  - transaction_id(PK) – Unique identifier of the savings transaction.
  - account_id(FK → Saving Account.account_id) – Account affected by the transaction.
  - transaction_type– Type of transaction (Deposit, Withdrawal, Interest posting, Fee Deduction).
  - amount– Amount of the transaction.
  - transaction_date– Date/time when the transaction occurred.
  - balance_after_transaction– Account balance immediately after this transaction is applied.
  - reference_number– External or internal reference (receipt, voucher, etc.).
  - employee_id(FK → Employee.employee_id) – Staff member who processed the transaction.

13. **Audit (Strong Entity):** Automatically tracks system actions performed by employees.

- Attributes
  - audit_id(PK) – Unique identifier of the audit record.
  - entity_name– Name of the table/entity modified (e.g., "Loan", "Member").
  - entity_id– Identifier of the specific record affected.
  - action_type– Type of action (Create, Update, Delete).
  - old_values– JSON payload with data snapshot before the change.
  - new_values– JSON payload with data snapshot after the change.
  - employee_id(FK → Employee.employee_id) – Staff user who made the change.
  - timestamp– Date/time of the action.
  - ip_address– Source IP address from which the change was made.

14. **Fee Type (Strong Entity):** A lookup entity that defines the types of fees available (e.g., Late Payment Fee, Processing Fee).

- Attributes
  - fee_type_id(PK) – Unique identifier of the fee type.
  - name– Fee type name (Loan Origination Fee, Late Payment Fee, Processing Fee, etc.).
  - description– Description of the fee and when it applies.
  - calculation_method– How the fee is calculated (Flat Amount, Percentage of Principal, Percentage of Due Amount).
  - amount_or_rate– Numeric amount or rate used in calculation.
  - is_active– Flag indicating whether this fee type is active.

15. **Fee Event (Strong Entity):** Captures an instance where a fee is triggered or generated by a loan activity or savings transaction.

- Attributes
    - fee_event_id(PK) – Unique identifier of the fee event.
    - fee_type_id(FK → Fee_Type.fee_type_id) – Type of fee.
    - member(FK → Member.member_id) – Member to whom this fee applies.
    - loan_id(FK → Loan.loan_id, nullable) – Loan associated with the fee (if loan-related).
    - saving_account_id(FK → Saving Account.account_id, nullable) – Saving account associated with the fee (if saving-related).

16. **Fee Transaction (Strong Entity):** Represents the actual financial posting/payment transaction of a generated fee.

- Attributes
    - fee_transaction_id(PK) – Unique identifier of the fee transaction.
    - fee_event(FK → Fee_Event.fee_event_id) – The fee event this transaction settles or affects.
    - amount– Amount paid or waived.
    - date– Date of the fee transaction.
    - reference– Status or reference (Pending, Paid, Waived, or free-text reference).

2.2.2 Relationship Specifications and Cardinalities

This section formally explains the structural rules linking entities via the diamonds seen in your ERD.

1.Branch — Employee (`Employs`)

- **Cardinality:** 1:N (One Branch employs many Employees).
- **Participation: Branch:** *Partial* (A newly registered or specialized digital branch might exist temporarily with zero recorded staff).
    - **Employee:** *Total* (Every employee must be systematically assigned to exactly one base branch).

2.Employee— Audit (`Performs Action`)

- **Cardinality:** 1:N (One Employee performs many Audited actions).
- **Participation:**
    - **Employee:** *Partial* (An employee might not have triggered any monitored, record-altering events yet).
    - **Audit:** *Total* (Every audit log must point to the specific actor/employee who performed it).

3.Employee — Loan (`Approves`)

- **Cardinality:** 1:N (One Employee reviews/approves multiple Loans).
- **Participation:**
    - **Employee:** *Partial* (Not all employees are loan officers; many will never approve a loan).
    - **Loan:** *Total* (Every loan requires a valid approving officer's signature/stamp to exist).

4.Employee — Saving Transaction (`ProcessSavingTransaction`)

- **Cardinality:** 1:N (One Employee / Teller processes multiple savings transactions).
- **Participation:**
    - **Employee:** *Partial* (Back-office employees don't run teller counters to process deposits).
    - **saving Transaction:** *Partial* (Automated system actions like digital interest postings or mobile deposits don't carry an explicit employee handler ID).

5.Branch — Employee (`Manages`)

- **Cardinality:** 1:1 (One Branch is managed by one Manager).
- **Participation:**
    - **Branch:** *Total* (Every open operational branch requires an appointed manager).
    - **Employee:** *Partial* (Most employees are regular staff, not branch managers).

6.Branch – Member (`Serves`)

- **Cardinality:** 1:N (One Branch registers/serves multiple Members).
- **Participation:**
    - **Branch:** *Partial* (A branch could open up in a new location before acquiring any members).
    - **Member:** *Total* (Every member must be mapped to a home/registering branch).

7.Branch — Saving Account (`Opened At`)

- **Cardinality:** 1:N (One Branch hosts many Savings Accounts).
- **Participation:**
    - **Branch:** *Partial* (A branch may open without having opened any accounts yet).
    - **Saving Account:** *Total* (An account must physically belong to the branch register where it was established).

8.Member — Saving Account (Owns Account)

- **Cardinality:** 1:N (One Member can hold multiple Savings Accounts).
- **Participation:**

- **Member:** *Partial* (A member could potentially join solely for loan access without saving).
- **Saving Account:** *Total* (A savings account cannot float freely; it must belong to an explicitly defined member).

9.Member — Loan (Takes Loan)

- **Cardinality:** 1:N (One Member can pull out multiple sequential or concurrent Loans).
- **Participation:**
  - **Member:** *Partial* (Many members use savings accounts exclusively and never apply for credit).
  - **Loan:** *Total* (Every loan file must link back to a valid borrower/member).

10.Member — Loan

- **Cardinality:** M:N (Many Members can act as guarantors across many Loans).
- **Participation:**
  - **Member:** *Partial* (A member is not forced to serve as a guarantor for others).
  - **Loan:** *Total* (Under this strict institutional rule, a loan product that requires a guarantor must tie to at least one member serving that function)
  -

11.Member ---Guarantor

Cardinality: N:M

Participation: Partial (Member) and Total **(**Guarantor**)**

Loan—Guarantor

- **Cardinality: N:M**
- **Participation:** Partial (Loan) and Total **(**Guarantor**)**

a. Loan Product — Loan (Defines Loan)

Cardinality: 1:N (One configured product governs many issued Loans).

Participation:

1. **Loan Product:** *Partial* (A newly designed loan product might sit on offer before a customer applies for it).
2. **Loan:** *Total* (Every active loan must explicitly inherit its ruleset from an official product template).

b. Saving Account Product — Saving Account (Configured By)

Cardinality: 1:N (One template product governs many active Savings Accounts).

Participation:

1. **Saving Account Product:** *Partial* (A product tier can exist without active users).
2. **Saving Account:** *Total* (Accounts must be built on top of valid product rules).

c. Loan — Collateral (Secured By)

cardinality: 1:N (One Loan can be secured by multiple items of collateral).
Participation:

1. **Loan:** *Partial* (Unsecured, small personal loan items don't require collateral backing).
2. **Collateral:** *Total* (An asset cannot be registered as collateral unless tied directly to an active, specific loan file).

d. Loan — Loan Schedule (`HasSchedule`)

Cardinality: 1:N (One Loan generates an entire multi-month payment schedule matrix).
Participation:

1. **Loan:** *Total* (Every single loan must have an amortized repayment timeline mapped immediately).
2. **Loan Schedule:** *Total* (Schedule lines cannot exist detached from a parent loan agreement).

e. Loan — Fee Event (`GeneratesLoanFee`)

Cardinality: 1:N (One Loan can prompt several distinct penalization/processing Fee Events).
Participation:

1. **loan:** *Partial* (A well-behaved member paying on time might never trigger additional fee events).
2. **Fee Event:** *Partial* (Because a fee event can be generated by alternative sources, like savings transactions).

f. Loan — Loan Transaction (`Has`)

Cardinality: 1:N (One Loan tracks a ledger history of many transactions).
Participation:

1. **Loan:** *Partial* (A loan file created right before accounting close might have 0 transactions briefly prior to disbursement).
2. **Loan Transaction:** *Total* (A payment entry must directly hit an explicit, existing loan balance).

g. Saving Account — Saving Transaction (`Has Saving Transaction`)

Cardinality: 1:N (One Savings Account logs an ongoing series of transactions).
Participation:

1. **Saving Account:** *Partial* (A freshly opened account could have a baseline 0.00 entry window prior to initial posting settlement).
2. **Saving Transaction:** *Total* (Every transaction must hit a real savings ledger).

h. Fee Type — Fee Event (`Defined By`)

Cardinality : 1:N (One standard policy Fee Type catalogs multiple triggered instances of Fee Events).

Participation:

1. **Fee Type:** *Partial* (A configured fee structure like "Foreign Check Reject Fee" may sit idle without getting triggered).
2. **Fee Event:** *Total* (Any system fee event must be explicitly categorized under an approved policy type).

i. Saving Transaction — Fee Event (`Generates Saving Transaction Fee`)

Cardinality: 1:N (One Saving Transaction can generate a Fee Event, such as an over-the-limit fee).

Participation:

1. **Saving Transaction:** *Partial* (Most normal savings transactions do not trigger extra fees).
2. **Fee Event:** *Partial* (Fee events can also be caused by Loans).

j. Fee Event — Fee Transaction (`HasFeeTransaction`)

Cardinality: 1:N (One Fee Event results in a downstream settlement Fee Transaction).

Participation:

1. **Fee Event:** *Partial* (The fee event might be waived or remain completely unpaid, meaning no payment transaction exists yet).
2. **Fee Transaction:** *Total* (Every fee payment entry must link back to an authorized, outstanding fee event invoice).

2.3 Logical / Relational Schema

The following logical relational schema is derived directly from the defined entities, attributes, keys, and relationship cardinalities of the SACCO Management System.

---

1. Branch

BRANCH(
branch_id PK,
name,
region,
city,
subcity,
kebele,
street,
phone_number,
manager_id FK → EMPLOYEE.employee_id UNIQUE,
created_date,
status

)

---

2. Employee

EMPLOYEE(
  employee_id PK,
  branch_id FK → BRANCH.branch_id,
  full_name,
  email,
  phone_number,
  role,
  hire_date,
  status
)

---

3. Member

MEMBER(
  member_id PK,
  branch_id FK → BRANCH.branch_id,
  full_name,
  gender,
  date_of_birth,
  national_id_number,
  kebele_id_number,
  email,
  phone_number,
  region,
  city,
  subcity,
  kebele,
  street,
  house_number,
  occupation,
  credit_score,
  membership_date,
  status
)

---

4. Loan Product

LOAN_PRODUCT(
  product_id PK,
  name,
  description,
  min_amount,
  max_amount,
  min_tenure_months,
  max_tenure_months,

base_interest_rate,
interest_type,
requires_guarantor,
requires_collateral,
status
)

---

5. Saving Account Product

SAVING_ACCOUNT_PRODUCT(
product_id PK,
name,
description,
min_balance,
interest_rate,
posting_frequency,
status
)

---

6. Saving Account

SAVING_ACCOUNT(
account_id PK,
member_id FK → MEMBER.member_id,
account_number,
branch_id FK → BRANCH.branch_id,
product_type FK → SAVING_ACCOUNT_PRODUCT.product_id,
current_balance,
open_date,
status

)

---

7. Loan

LOAN(
loan_id PK,
member_id FK → MEMBER.member_id,
product_id FK → LOAN_PRODUCT.product_id,
loan_officer_id FK → EMPLOYEE.employee_id,
principal_amount,
interest_rate,
term_months,
disbursement_date,
maturity_date,
outstanding_principal,
outstanding_interest,
outstanding_fees,
status
)

8. Guarantor (Associative Entity)

GUARANTOR(
loan_id PK FK → LOAN.loan_id,
member_id PK FK → MEMBER.member_id,
guarantee_amount,
status
)

9. Collateral

COLLATERAL(
collateral_id PK,
loan_id FK → LOAN.loan_id,
collateral_type,
description,
estimated_value,
ownership_document_ref,
status
)

10. Loan Schedule

LOAN_SCHEDULE(
schedule_id PK,
loan_id FK → LOAN.loan_id,
installment_number,
due_date,
principal_due,
interest_due,
fees_due,
total_due,
principal_paid,
interest_paid,
fees_paid,
status
)

11. Loan Transaction

LOAN_TRANSACTION(
transaction_id PK,
loan_id FK → LOAN.loan_id,
transaction_type,
amount,
transaction_date,
payment_method,
reference_number,
employee_id FK → EMPLOYEE.employee_id,

reversal_status
)

---

12. Saving Transaction

SAVING_TRANSACTION(
transaction_id PK,
account_id FK → SAVING_ACCOUNT.account_id,
transaction_type,
amount,
transaction_date,
balance_after_transaction,
reference_number,
employee_id FK → EMPLOYEE.employee_id
)

---

13. Audit

AUDIT(
audit_id PK,
entity_name,
entity_id,
action_type,
old_values,
new_values,
employee_id FK → EMPLOYEE.employee_id,
timestamp,
ip_address
)

---

14. Fee Type

FEE_TYPE(
fee_type_id PK,
name,
description,
calculation_method,
amount_or_rate,
is_active
)

---

15. Fee Event

FEE_EVENT(
fee_event_id PK,
fee_type_id FK → FEE_TYPE.fee_type_id,
member_id FK → MEMBER.member_id,
loan_id FK → LOAN.loan_id NULL,
saving_account_id FK → SAVING_ACCOUNT.account_id NULL
)

16. Fee Transaction

FEE_TRANSACTION(
fee_transaction_id PK,
fee_event_id FK → FEE_EVENT.fee_event_id,
amount,
date, reference)

2.4 RELATIONAL DIAGRAM

This is the **relational diagram** of the Yisakal SACCO management system. It illustrates the structure of the database by showing the tables, their attributes, primary keys, and the relationships between them using foreign keys. The diagram represents how different SACCO entities such as branches, employees, members, saving accounts, loans, transactions, guaranties, collateral, fees, and audit records are connected within the system. It helps to ensure data integrity, reduce redundancy, and support efficient management of SACCO operations.

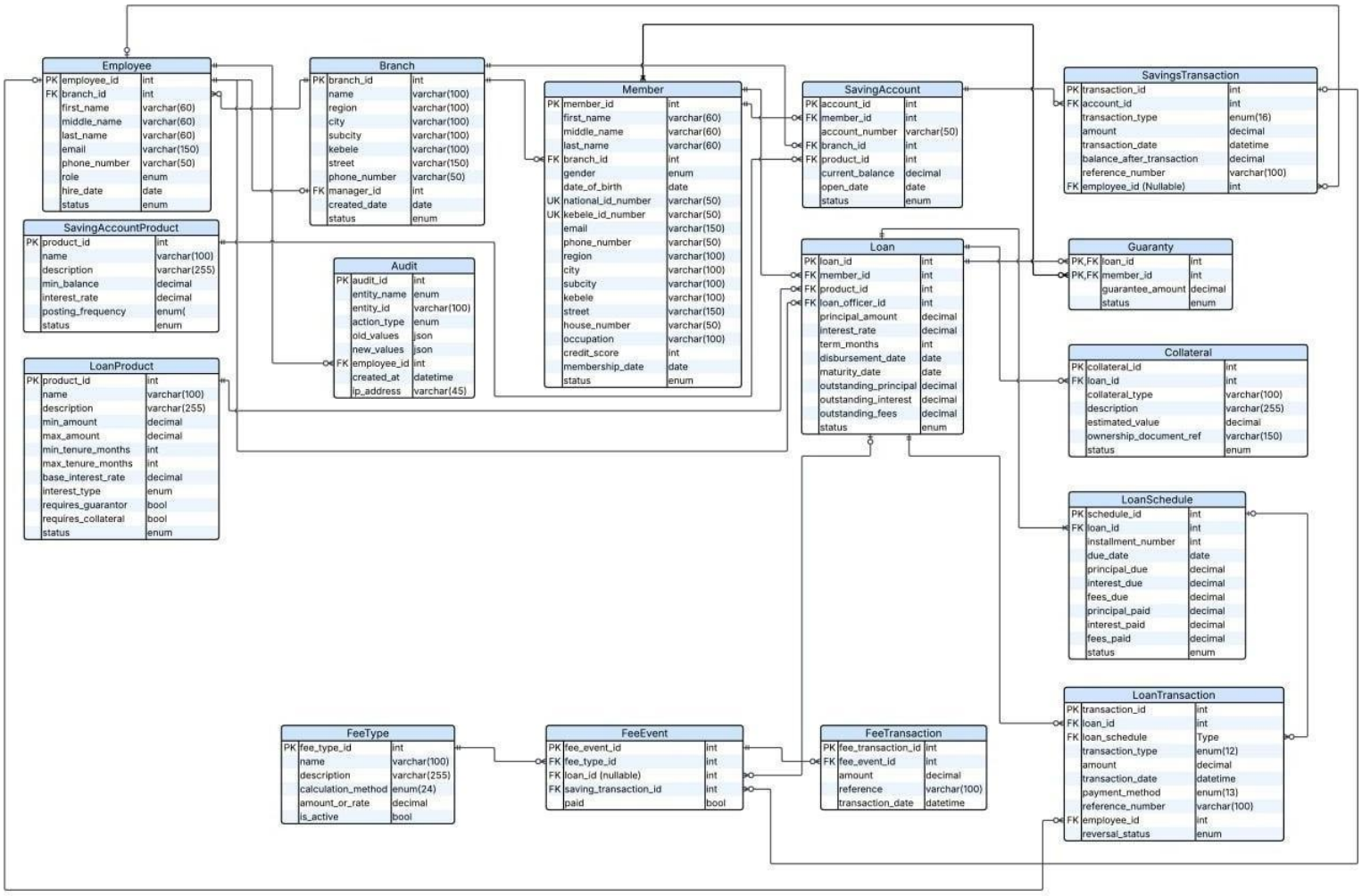


Figure 2: Relational ER diagram of YISAKAL SACCO

2.5 YISAKAL SACCO DATABASE NORMALIZATION ANALYSIS

(*A step by step normalization to Boyce-Codd Normal Form (BCNF))*

This normalization analysis document presents how the data is modeled and is normalized by considering the structural entities and relationships defined in the conceptual ERD. It follows series of steps followed to obtain a database design that allows for consistent storage and efficient access of data in a relational database. These steps reduce data redundancy and the risk of data becoming inconsistent.

- In order to obtain a normalized form to BCNF, we must follow the following steps:-
  - **Step 1: Create Unnormalized Data:-** Construct a wide, un-atomic, messy relation

combining highly integrated metrics to simulate raw application states and serve as the baseline for structurally organizing it.

- **Step 2: Convert to 1st Normal Form (1NF):-** Eliminate repeating data collections, cross-product records, and multi-valued string lists. It enforces strict structural atomicity.
- **Step 3:- Convert to 2nd Normal Form (2NF)**:- Identify and remove all partial dependencies. Ensure that every non-prime attribute exhibits full functional dependency on the designated primary key framework.
- **Step 4:- Convert to 3rd Normal Form (3NF)**:- Identify and isolate all transitive dependency chains. Ensure that no non-prime structural element relies functionally on another non-prime attribute.
- **Step 5**:- **Convert to Boyce-Codd Normal Form (BCNF)**: Perform final determinant verification. Ensure that for every non-trivial functional dependency $X \rightarrow Y$, the left-hand side determinant X constitutes a valid, indisputable super key or candidate key.

PART 1: Administrative location, Branch and Member Subsystem

Step 1:- Create unnormalized data

The unnormalized relation consists customer demographics, variable multi-valued contact entries, geographical elements, and operational parent branch office definitions inside a single flat table grid.

member_id	full_name	branch_id	national_id	phone_number	Kebele	subcity	city	region	branch_name
M-901	Abebe Kebede Balcha	BR-001	NID-A9901	+25191111, +25192222	03	Bole	A.A	A.A	Bole branch
M-902	Chaltu Hunde Tafa	BR-001	NID-O2241	+25191144	01	Sebeta	Sebeta	Oromia	Sebeta branch

Step 2:-

Convert to 1NF:-

The row attributes display singular structural atomicity. Multi-valued lists are avoided since individual member and branch phone contact records are maintained within single-value attributes matching the operational system layout..

member_id	full_name	branch_id	national_id	phone_number	Kebele	subcity	city	region	branch_name
M-901	Abebe Kebede Balcha	BR-001	NID-A9901	+25191111,	03	Bole	A.A	A.A	Bole branch
M-901	Abebe Kebede Balcha	BR-001	NID-A9901	+25192222	03	Bole	A.A	A.A	Bole branch
M-902	Chaltu Hunde Tafa	BR-001	NID-O2241	+25191144	01	Sebeta	Sebeta	Oromia	Sebeta branch

Initial Composite Primary Key: (member_id, branch_id)

FD 1.1: (member_id, branch_id) → full_name, national_id, kebele, subcity, city, region, phone_number ,branch_name***(Full Key Dependency)***

FD 1.2: branch_id → branch_name, region, city, subcity,kebele***(Partial Key Dependency)***

FD 1.3: member_id→ full_name, branch_id , national_id, region, city, subcity,kebele, phone_number, ***(Alternative Unique Natural Key)***

Determinant)

Step 3:- Convert to 2NF:- We isolate attributes that suffer from partial dependency on the subset identifier member_id away from the multivalued phone number string tracker.

- **2NF_BRANCH** (branch_id, branch_name, region, city, subcity, kebele)
- **2NF_MEMBER** (member_id, full_name, branch_id, national_id, kebele, phone_number, region, city, subcity)

STEP 4 — Convert to 3NF

Reviewing the schemas reveals that geographical attributes are functionally dependent directly on the specific entity IDs (member_id and branch_id) as explicit attributes of their physical

- **3NF_BRANCH** (branch_id, branch_name, region, city, subcity, kebele)
- **3NF_MEMBER** (member_id, full_name, branch_id, national_id, kebele, phone_number, region, city, subcity)

definitions within this specific operational requirement. Transitive dependency analysis checks pass cleanly for both relations.

Step 5 — Convert to BCNF

We audit the remaining functional determinants. In 3NF_MEMBER, member_id, national_id_number, and kebele_id_number serve as valid candidate keys. In 3NF_BRANCH, branch_id is the absolute determinant. Every determinant matches a superkey profile exactly.

- **Branch** (branch_id, branch_name, region, city, subcity, kebele)
- **Member** (member_id, full_name, branch_id, gender, date_of_birth, national_id, phone_number, region, city, subcity, kebele,)

Part 2: Internal Human Resources and Audit log framework

STEP 1 — Create UNNORMALIZED DATA (Employee-Audit table)

This relation links employee identities, functional system access security roles, and specific transactional operations historical metadata sequences into a single table.

employee_id	employee full_name	branch_id	phone_number	audit_id	action_type	timestamp	old_values	new_values
E-01	Almaz Tekle Haile	BR-001	+25191111,	AUD-901	APPROVE_LOAN	2026-05-20 10:15	{“status”: “Pending”}	{“status”: “Approved”}
E-02	Almaz Tekle Balcha	BR-001	+25192222	AUD-942	UPDATE_MEMBER	2026-05-20 11:30	{“house_no”: “4555”}	{“house_no”: “6555”}
E-03	Bekele Zewdu Assefa	BR-001	+25191144	AUD-955	POST_DEPOSIT	2026-05-20 11:45	{“balance”: “20,500”}	{“balance”: “26,780”}

Step 2 — Convert to 1NF

The row attributes display singular structural atomicity. No multivalued data elements exist inside individual cells.

Initial Composite Primary Key: (employee_id, audit_id)

FD 2.1: (employee_id, audit_id) → employeefull_name, branch_id, action_type, timestamp, old_values, new_values ***(Full Key Dependency)***

FD 2.2: employee_id → employee_name, branch_id ***(Partial Key Dependency Violation)***

FD 2.3: audit_id → employee_id, action_type, timestamp, old_values, new_values ***(Partial Key Dependency Violation)***

STEP 3 — Convert to 2NF

Because ***employee_name*** and ***action_type*** rely entirely on independent component fractions of the initial key string, we break the relation down to satisfy 2NF compliance rules.

2NF_EMPLOYEE (**employee_id**, employee_name, branch_id)

2NF_AUDIT_LOG (**audit_id**, employee_id, action_type, timestamp, old_values, new_values)

STEP 4 — Convert to 3NF

Reviewing the tables reveals that no non-key fields determine other non-key values. Therefore it passes transitive dependency analysis for both relations.

STEP 5 — Convert to BCNF

Every functional dependency determinant matches an explicitly assigned unique table primary key. No further functional structural transformation is required.

Employee (**employee_id**, employee_name, role**,** *branch_id***)**

Audit_Log (**audit_id**, *employee_id***,** action_type, timestamp, old_values, new_values)

Part 3: Financial Products, Saving Accounts and Transactions

Step 1:- Create unnormalized data

This table has interest products configuration constraints, customer savings values, and linear

Step 2: Convert to 1st Normal Form (1NF)

account_id	account_no	member_id	product_id	product_name	interest_rate	transaction_id	amount	Balance_after_transaction
ACC-501	100023455	M-901	PRD-SAV	Deposit	5.00	TX-8801,TX-8802	5000	25500.00
ACC-502	100023444	M-902	PRD-SAV	Deposit	5.00	TX-8803	7000	34500.00

Every attribute cell holds strictly atomic data. The dynamic collection of attributes is controlled by a composite primary key layout.

- **Primary Key:** (account_id, product_id, transaction_id)
- **Functional Dependencies (FDs):**
  - **FD 3.1 (Composite Key Link):** (account_id, product_id, transaction_id) ->, amount, balance_after_transaction, account_number, interest_rate
  - **FD 3.2 (Account Cluster):** account_id -> account_number, product_id, member_id, branch_id *(Partial Key Dependency Violation)*
  - **FD 3.3 (Product Settings):** product_id -> product_name, interest_rate*(Partial Key Dependency Violation)*
  - **FD 3.4 (Transaction Event):** transaction_id -> account_id, amount, balance_after_transaction,*(Partial Key Dependency Violation)*

STEP 3: Convert to 2nd Normal Form (2NF)

We remove the partial dependencies identified in Step 2 by isolating attributes that depend on only a subset of the composite key into independent tables.

- **2NF_SAVING_PRODUCT** (product_id, product_name, interest_rate)
- **2NF_SAVING_ACCOUNT** (account_id, account_number, product_id, member_id, branch_id)
- **2NF_SAVINGS_TRANSACTION** (transaction_id, account_id, amount, balance_after_transaction)

STEP 4: Convert to 3rd Normal Form (3NF)

We check for transitive dependencies where a non-prime attribute determines another non-prime attribute (X -> Y ->Z). Within these isolated boundaries, no transitively linked elements exist; settings rely directly on their respective key roots.

- **3NF_SAVING_PRODUCT** (product_id, product_name, interest_rate)
- **3NF_SAVING_ACCOUNT** (account_id, account_number, product_id, member_id, branch_id)
- **3NF_SAVINGS_TRANSACTION** (transaction_id, account_id, amount, balance_after_transaction)

STEP 5: Convert to Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if, for every non-trivial functional dependency X -> Y ; X is a superkey. In 3NF_SAVING_ACCOUNT, account_number is unique and acts as a valid secondary candidate key (account_number -> account_id...). Because every active left-hand determinant represents a verified superkey, the relations are confirmed in BCNF.

- **SavingAccountProduct** (product_id [PK], product_name, interest_rate)
- **SavingAccount** (account_id [PK], account_number [SK], product_id [FK], member_id, branch_id)
- **SavingsTransaction** (transaction_id [PK], account_id [FK], amount, balance_after_transaction)

Part 4: CAPITAL LOANS, REPAYMENT SCHEDULES, AND ASSET COLLATERAL MAPPING

STEP 1: Create UNNORMALIZED DATA

This table combines active loan commitments, targeted guarantor financial limits, and asset risk records into a single table.

loan_id	principal_amount	schedule_id	installment_number	collateral_id	ownership_document_ref	guarantor_member_id	guarantee_amount
LN-800	500000	SCH-01	01	COL-55	REF-VEH99	M-905	200000
LN-801	750000	SCH-02	02	COL-56	REF-PROP4	M-906	500000

STEP 2: Convert to 1st Normal Form (1NF)

To make the data easier to organize in tables, attributes that can have many values are separated into simpler values. Since each record contains combined information, a larger primary key made from several attributes is used to uniquely identify each row without duplication

- **Primary Key:** (loan_id, schedule_id, collateral_id, guarantor_member_id)
- **Functional Dependencies (FDs):**
    - **FD 4.1 (Loan Master):** loan_id -> principal_amount, product_id, , term_months*(Partial Key Dependency Violation)*
    - **FD 4.2 (Amortization Line):** schedule_id -> loan_id, installment_number, due_date *(Partial Key Dependency Violation)*
    - **FD 4.3 (Secured Collateral):** collateral_id -> loan_id, ownership_document_ref, estimated_value, status *(Partial Key Dependency Violation; maps 1:M to Loan directly per the schema specification*
    - **Guaranty Matrix):** (loan_id, guarantor_member_id) -> guarantee_amount, status *(Partial Key Dependency Violation)*

STEP 3: Convert to 2nd Normal Form (2NF)

We systematically eliminate the structural partial dependencies verified in Step 2 by isolating each dependent cluster into independent, focused tables.

2NF_LOAN

loan_id	**principal_amount**	**product_id**	**term_months**
LN-880	500000	LP-01	12 Months
LN-881	750000	LP-02	24 Months

2NF_LOAN_SCHEDULE

schedule_id	loan_id	installment_number	due_date
SCH-01	LN-880	01	2026-03-01
SCH-02	LN-881	02	2026-04-15

2NF_COLLATERAL

collateral_id	loan_id	ownership_document_status	estimated_value	status
COL-55	LN-880	REF-VEH99	900000	Pledge
COL-56	LN-881	REF-PROP4	150000	Pledge

2NF_GUARANTY

loan_id	guarantor_member_id	guarantee_amount	status
LN-880	M-905	200000	Active
LN-881	M-906	350000	Active

STEP 4: Convert to 3rd Normal Form (3NF)

We audit the 2NF structures for transitive dependencies (X-> Y-> Z) where non-prime attributes functionally determine other non-prime elements. Within these four isolated limits, all attributes depend directly on their respective key roots without intermediate links.

- **3NF_LOAN** (Same attributes as 2NF_LOAN)
- **3NF_LOAN_SCHEDULE** (Same attributes as 2NF_LOAN_SCHEDULE)
- **3NF_COLLATERAL** (Same attributes as 2NF_COLLATERAL)
- **3NF_GUARANTY** (Same attributes as 2NF_GUARANTY)

STEP 5: Convert to Boyce-Codd Normal Form (BCNF)

A relation satisfies BCNF if every determinant is a superkey. In 3NF_COLLATERAL, the unique registration reference field forms a secondary candidate key (ownership_document_ref -> collateral_id, loan_id...). Because every non-trivial dependency determinant represents an absolute superkey, BCNF is met cleanly.

- **Loan** (loan_id [PK], principal_amount, product_id, loan_officer_id, term_months)
- **LoanSchedule** (schedule_id [PK], loan_id [FK], installment_number, due_date)
- **Collateral** (collateral_id [PK], loan_id [FK], ownership_document_ref [SK], estimated_value, status)
- **Guaranty** ((loan_id, guarantor_member_id) [PK/FK], guarantee_amount, status)

Part 5: SYSTEM FEE RULES, CHARGING FRAMEWORK, AND EXECUTION INSTANCES

STEP 1: Create UNNORMALIZED DATA

This processing layer captures execution log instances, operational link triggers (loans vs. transactional accounts), and the system configuration properties that calculate fees.

fee_ transaction_ id	amount	fee_ event _id	loan_ id	saving_ transaction_ id	fee_ type_ id	calculation_ method	amount
FT-11	150.00	EV-101	LN-880	TX-8801	FT-PERC	Percentage	1.50
FT-12	700.00	EV-102	LN-881	TX-8899	FT-FLAT	Flat	50.00

STEP 2: Convert to 1st Normal Form (1NF)

All entries are simplified into clear, atomic cell components.

- **Initial Primary Key:** fee_transaction_id
- **Functional Dependencies (FDs):**
  - **FD 5.1:** fee_transaction_id -> fee_event_id, amount, reference, transaction_date
  - **FD 5.2:** fee_event_id -> fee_type_id, loan_id, saving_transaction_id, paid
  - **FD 5.3:** fee_type_id -> name, description, calculation_method, amount_or_rate, is_active *(Transitive Linkage)*

STEP 3: Convert to 2nd Normal Form (2NF)

Because the unnormalized baseline relation relies on a single-column primary key identifier (fee_transaction_id), partial key functional dependencies are mathematically impossible. The table satisfies 2NF parameters natively.

STEP 4: Convert to 3rd Normal Form (3NF)

The schema exhibits transitive dependency chains where the primary identifier dictates a non-prime attribute, which in turn determines other non-prime values (fee_transaction_id -> fee_event_id -> fee_type_id). We resolve these anomalies by decomposing the structure into three distinct tables.

3NF_FEE_TYPE

fee_type_id	name	description	calculation_method	amount_or_rate	is_active

FT-PERC	Processing Fee	Standard	Percentage	1.50	true
FT-FLAT	ATM Withdrawal Fee	Penalty	Flat	50.00	true

3NF_FEE_EVENT

fee_event_id	fee_type_id	loan_id	saving_transaction_id	paid
EV-101	FT-PERC	LN-880	TX-8801	false
EV-102	FT-FLAT	LN-881	TX-8899	true

3NF_FEE_TRANSACTION

fee_transaction_id	fee_event_id	amount	reference	transaction_date
FT-11	EV-101	150.00	REF-FEE01	2026-05-20
FT-12	EV-102	50.00	REF-FEE02	2026-05-21

STEP 5: Convert to Boyce-Codd Normal Form (BCNF)

We audit the remaining functional dependencies. Every active left-hand determinant matches the assigned primary key superkey for its respective table, confirming full compliance with BCNF standards.

- **FeeType** (fee_type_id [PK], name, description, calculation_method, amount_or_rate, is_active)
- **FeeEvent** (fee_event_id [PK], fee_type_id [FK], loan_id, saving_transaction_id, paid)
- **FeeTransaction** (fee_transaction_id [PK], fee_event_id [FK], amount, reference, transaction_date)

Comprehensive Verified Candidate Key Super-Matrix

This verification matrix confirms that every mapped table directly aligns with your physical SQL design while operating cleanly under Boyce-Codd Normal Form (BCNF) requirements.

Target Table Name	Primary Key (PK)	Candidate Keys (CK) / Determinants	BCNF Compliance Status
Branch	branch_id	branch_id	CONFIRMED (Superkey)
Employee	employee_id	employee_id	CONFIRMED (Superkey)
Member	member_id	member_id, national_id_number, kebele_id_number	CONFIRMED (Superkey)
SavingAccountProduct	product_id	product_id	CONFIRMED (Superkey)
LoanProduct	product_id	product_id	CONFIRMED (Superkey)
SavingAccount	account_id	account_id, account_number	CONFIRMED (Superkey)
Loan	loan_id	loan_id	CONFIRMED (Superkey)
LoanSchedule	schedule_id	schedule_id	CONFIRMED (Superkey)
SavingsTransaction	transaction_id	transaction_id	CONFIRMED (Superkey)
LoanTransaction	transaction_id	transaction_id	CONFIRMED (Superkey)
Guaranty	(loan_id, member_id)	(loan_id, member_id)	CONFIRMED (All keys prime)
Collateral	collateral_id	collateral_id, ownership_document_ref	CONFIRMED (Superkey)
Audit	audit_id	audit_id	CONFIRMED (Superkey)
FeeType	fee_type_id	fee_type_id	CONFIRMED (Superkey)
FeeEvent	fee_event_id	fee_event_id	CONFIRMED (Superkey)
FeeTransaction	fee_transaction_id	fee_transaction_id	CONFIRMED (Superkey)

Chapter 3: Implementation

In this phase, the system is implemented using both MySQL and MongoDB. It involves creating tables and collections, inserting sample data, and performing queries to test the functionality of the system. The results are documented together with any identified bugs and suggested improvements to enhance performance and reliability.

3.1 MySQL Implementation

The MySQL part of the system is implemented using relational tables to represent SACCO entities such as Branch, Employee, Member, Loan, SavingAccount, transactions, fees, collateral, and audit records.
Each table is created with appropriate attributes, primary keys, and foreign key relationships to maintain data integrity and ensure proper relationships between entities. The system uses constraints such as NOT NULL, UNIQUE, ENUM, and FOREIGN KEY to enforce data validity.
After creating the tables, sample data is inserted into each table using INSERT statements. This data simulates real SACCO operations such as member registration, loan issuance, savings deposits, repayments, fee processing, and auditing activities.

```
CREATE DATABASE Yisakal_SACCO;
USE Yisakal_SACCO;

CREATE TABLE Branch (
branch_id int PRIMARY KEY AUTO_INCREMENT,
name varchar(100) NOT NULL,
region varchar(100),
city varchar(100),
subcity varchar(100),
kebele varchar(100),
street varchar(150),
phone_number varchar(50),
manager_id int,
created_date date DEFAULT (CURRENT_DATE),
status enum('Active', 'Inactive') DEFAULT 'Active'
);

CREATE TABLE Employee (
employee_id int PRIMARY KEY AUTO_INCREMENT,
branch_id int,
full_name varchar(150) NOT NULL,
```

```
email varchar(150),
phone_number varchar(50),
role enum('Admin', 'Manager', 'Officer', 'Teller') NOT NULL,
hire_date date DEFAULT (CURRENT_DATE),
status enum('Active', 'Inactive', 'Terminated') DEFAULT 'Active',
FOREIGN KEY (branch_id)
REFERENCES Branch(branch_id)
);

CREATE TABLE Member (
member_id int PRIMARY KEY AUTO_INCREMENT,
full_name varchar(150) NOT NULL,
branch_id int,
gender enum('Male', 'Female'),
date_of_birth date,
national_id_number varchar(50),
kebele_id_number varchar(50),
email varchar(150),
phone_number varchar(50),
region varchar(100),
city varchar(100),
subcity varchar(100),
kebele varchar(100),
street varchar(150),
house_number varchar(50),
occupation varchar(100),
credit_score int,
membership_date date DEFAULT (CURRENT_DATE),
status enum('Active', 'Inactive', 'Suspended') DEFAULT 'Active',
FOREIGN KEY (branch_id)
REFERENCES Branch(branch_id),
UNIQUE KEY UK_national_id (national_id_number),
UNIQUE KEY UK_kebele_id (kebele_id_number)
);

CREATE TABLE SavingAccountProduct (
product_id int PRIMARY KEY AUTO_INCREMENT,
name varchar(100) NOT NULL,
description varchar(255),
min_balance decimal(18,2) DEFAULT 0.00,
interest_rate decimal(5,2) NOT NULL,
posting_frequency enum('Daily', 'Weekly', 'Monthly', 'Yearly') NOT NULL,
status enum('Active', 'Inactive') DEFAULT 'Active'
);

CREATE TABLE LoanProduct (
product_id int PRIMARY KEY AUTO_INCREMENT,
name varchar(100) NOT NULL,
```

```
description varchar(255),
min_amount decimal(18,2),
max_amount decimal(18,2),
min_tenure_months int,
max_tenure_months int,
base_interest_rate decimal(5,2),
interest_type enum('Flat', 'Reducing') NOT NULL,
requires_guarantor bool DEFAULT false,
requires_collateral bool DEFAULT false,
status enum('Active', 'Inactive') DEFAULT 'Active'
);

CREATE TABLE SavingAccount (
account_id int PRIMARY KEY AUTO_INCREMENT,
member_id int NOT NULL,
account_number varchar(50) UNIQUE NOT NULL,
branch_id int,
product_id int NOT NULL,
current_balance decimal(18,2) DEFAULT 0.00,
open_date date DEFAULT (CURRENT_DATE),
status enum('Active', 'Inactive', 'Closed') DEFAULT 'Active',
FOREIGN KEY (product_id)
REFERENCES SavingAccountProduct(product_id),
FOREIGN KEY (member_id)
REFERENCES Member(member_id),
FOREIGN KEY (branch_id)
REFERENCES Branch(branch_id)
);

CREATE TABLE Loan (
loan_id int PRIMARY KEY AUTO_INCREMENT,
member_id int NOT NULL,
product_id int NOT NULL,
loan_officer_id int,
principal_amount decimal(18,2) NOT NULL,
interest_rate decimal(5,2) NOT NULL,
term_months int NOT NULL,
disbursement_date date,
maturity_date date,
outstanding_principal decimal(18,2),
outstanding_interest decimal(18,2),
outstanding_fees decimal(18,2),
status enum('Applied', 'Approved', 'Disbursed', 'Active', 'Closed', 'Defaulted')
DEFAULT 'Applied',
FOREIGN KEY (member_id)
REFERENCES Member(member_id),
FOREIGN KEY (loan_officer_id)
REFERENCES Employee(employee_id),
```

```
FOREIGN KEY (product_id)
REFERENCES LoanProduct(product_id)
);
CREATE TABLE LoanSchedule (
schedule_id int PRIMARY KEY AUTO_INCREMENT,
loan_id int NOT NULL,
installment_number int NOT NULL,
due_date date NOT NULL,
principal_due decimal(18,2) DEFAULT 0.00,
interest_due decimal(18,2) DEFAULT 0.00,
fees_due decimal(18,2) DEFAULT 0.00,
principal_paid decimal(18,2) DEFAULT 0.00,
interest_paid decimal(18,2) DEFAULT 0.00,
fees_paid decimal(18,2) DEFAULT 0.00,
status enum('Pending', 'Paid', 'Partial', 'Overdue') DEFAULT 'Pending',
FOREIGN KEY (loan_id)
REFERENCES Loan(loan_id)
);

CREATE TABLE SavingsTransaction (
transaction_id int PRIMARY KEY AUTO_INCREMENT,
account_id int NOT NULL,
transaction_type enum('Deposit', 'Withdrawal', 'Interest', 'Fee') NOT NULL,
amount decimal(18,2) NOT NULL,
transaction_date datetime DEFAULT CURRENT_TIMESTAMP,
balance_after_transaction decimal(18,2),
reference_number varchar(100),
employee_id int,
FOREIGN KEY (account_id)
REFERENCES SavingAccount(account_id),
FOREIGN KEY (employee_id)
REFERENCES Employee(employee_id)
);

CREATE TABLE LoanTransaction (
transaction_id int PRIMARY KEY AUTO_INCREMENT,
loan_id int NOT NULL,
loan_schedule_id int,
transaction_type enum('Disbursement', 'Repayment', 'Interest', 'Fee') NOT NULL,
amount decimal(18,2) NOT NULL,
transaction_date datetime DEFAULT CURRENT_TIMESTAMP,
payment_method enum('Cash', 'Transfer', 'Check') NOT NULL,
reference_number varchar(100),
employee_id int,
reversal_status enum('None', 'Reversed', 'Correction') DEFAULT 'None',
FOREIGN KEY (loan_id)
REFERENCES Loan(loan_id),
FOREIGN KEY (employee_id)
```

```
REFERENCES Employee(employee_id),
FOREIGN KEY (loan_schedule_id)
REFERENCES LoanSchedule(schedule_id)
);

CREATE TABLE Guaranty (
loan_id int,
member_id int,
guarantee_amount decimal(18,2) NOT NULL,
status enum('Active', 'Released', 'Claimed') DEFAULT 'Active',
PRIMARY KEY (loan_id, member_id),
FOREIGN KEY (loan_id)
REFERENCES Loan(loan_id),
FOREIGN KEY (member_id)
REFERENCES Member(member_id)
);

CREATE TABLE Collateral (
collateral_id int PRIMARY KEY AUTO_INCREMENT,
loan_id int NOT NULL,
collateral_type varchar(100) NOT NULL,
description varchar(255),
estimated_value decimal(18,2),
ownership_document_ref varchar(150),
status enum('Pledged', 'Released', 'Liquidated') DEFAULT 'Pledged',
FOREIGN KEY (loan_id)
REFERENCES Loan(loan_id)
);

CREATE TABLE Audit (
audit_id int PRIMARY KEY AUTO_INCREMENT,
entity_name enum('Member', 'Loan', 'SavingAccount', 'Employee', 'Branch') NOT
NULL,
entity_id varchar(100) NOT NULL,
action_type enum('Create', 'Update', 'Delete', 'Login') NOT NULL,
old_values json,
new_values json,
employee_id int,
created_at datetime DEFAULT CURRENT_TIMESTAMP,
ip_address varchar(45),
FOREIGN KEY (employee_id)
REFERENCES Employee(employee_id)
);

CREATE TABLE FeeType (
fee_type_id int PRIMARY KEY AUTO_INCREMENT,
name varchar(100) NOT NULL,
description varchar(255),
```

```
calculation_method enum('Flat', 'Percentage') NOT NULL,
amount_or_rate decimal(18,2) NOT NULL,
is_active bool DEFAULT true
);

CREATE TABLE FeeEvent (
fee_event_id int PRIMARY KEY AUTO_INCREMENT,
fee_type_id int NOT NULL,
loan_id int,
saving_transaction_id int,
paid bool DEFAULT false,
FOREIGN KEY (saving_transaction_id)
REFERENCES SavingsTransaction(transaction_id),
FOREIGN KEY (fee_type_id)
REFERENCES FeeType(fee_type_id),
FOREIGN KEY (loan_id)
REFERENCES Loan(loan_id)
);

CREATE TABLE FeeTransaction (
fee_transaction_id int PRIMARY KEY AUTO_INCREMENT,
fee_event_id int NOT NULL,
amount decimal(18,2) NOT NULL,
reference varchar(100),
transaction_date datetime DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (fee_event_id)
REFERENCES FeeEvent(fee_event_id)
);

ALTER TABLE Branch
ADD FOREIGN KEY (manager_id)
REFERENCES Employee(employee_id);
```

This section presents the input values entered into the SQL queries and the corresponding results generated from the database during system testing and implementation.

```
-- BRANCH
INSERT INTO Branch (branch_id, name, region, city, subcity, kebele, street, phone_number, created_date,
status) VALUES
(1,'Merkato Branch','Addis Ababa','Addis Ababa','Addis Ketema','05','Merkato Ave','0111234567','2020-01-
15','Active'),
(2,'Bole Branch','Addis Ababa','Addis Ababa','Bole','03','Bole Rd','0119876543','2020-06-01','Active'),
(3,'Bahir Dar Branch','Amhara','Bahir Dar','Fasilo','01','Lake Tana St','0581234567','2021-03-10','Active'),
(4,'Jimma Branch','Oromia','Jimma','Jimma','02','Main St','0471112233','2021-08-12','Active'),
(5,'Hawassa Branch','SNNPR','Hawassa','Tabor','04','Unity St','0462223344','2022-02-10','Active');
```

```
-- EMPLOYEE
INSERT INTO Employee (employee_id, branch_id, full_name, email, phone_number, role, hire_date, status)
VALUES
(101,1,'Abebe Kebede','abebe@yisakal.com','0911111111','Manager','2020-01-20','Active'),
(102,1,'Sara Tesfaye','sara@yisakal.com','0922222222','Officer','2020-03-15','Active'),
(103,2,'Dawit Haile','dawit@yisakal.com','0933333333','Teller','2021-01-10','Active'),
(104,2,'Meron Alemu','meron@yisakal.com','0944444444','Manager','2020-06-05','Active'),
(105,3,'Yonas Girma','yonas@yisakal.com','0955555555','Officer','2021-04-01','Active');

-- MEMBER
INSERT INTO Member (member_id, full_name, branch_id, gender, date_of_birth, national_id_number,
kebele_id_number, email, phone_number, region, city, subcity, kebele, street, house_number, occupation,
credit_score, membership_date, status) VALUES
(201,'Tigist Bekele',1,'Female','1990-05-12','NID1001','KID1001','tigist@gmail.com','0966666666','Addis
Ababa','Addis Ababa','Addis Ketema','05','Merkato Ave','12A','Trader',720,'2021-01-10','Active'),
(202,'Kebede Desta',1,'Male','1985-11-03','NID1002','KID1002','kebede@gmail.com','0977777777','Addis
Ababa','Addis Ababa','Arada','02','Piassa Rd','5','Teacher',680,'2021-02-20','Active'),
(203,'Helen Tadesse',2,'Female','1992-08-25','NID1003','KID1003','helen@gmail.com','0988888888','Addis
Ababa','Addis Ababa','Bole','07','Cameroon St','33','Nurse',750,'2021-06-15','Active'),
(204,'Solomon Gebre',2,'Male','1988-02-14','NID1004','KID1004','solomon@gmail.com','0912345678','Addis
Ababa','Addis Ababa','Yeka','10','Megenagna Rd','8B','Engineer',790,'2022-01-05','Active'),
(205,'Fatuma Ahmed',3,'Female','1995-12-
30','NID1005','KID1005','fatuma@gmail.com','0923456789','Amhara','Bahir Dar','Fasilo','01','University
Ave','15','Accountant',710,'2022-03-20','Active');

-- SAVING ACCOUNT PRODUCT
INSERT INTO SavingAccountProduct
(product_id,name,description,min_balance,interest_rate,posting_frequency,status) VALUES
(301,'Regular Savings','Standard account',100,7,'Yearly','Active'),
(302,'Youth Savings','Youth account',50,8.5,'Monthly','Active'),
(303,'Fixed Deposit','12 months deposit',5000,10,'Yearly','Active'),
(304,'Business Savings','Business account',1000,9,'Monthly','Active'),
(305,'Women Savings','Women empowerment account',200,8,'Monthly','Active');

-- LOAN PRODUCT
INSERT INTO LoanProduct
(product_id,name,description,min_amount,max_amount,min_tenure_months,max_tenure_months,base_interest_rate,in
terest_type,requires_guarantor,requires_collateral,status) VALUES
(401,'Personal Loan','Personal needs',5000,100000,6,36,15,'Reducing',true,false,'Active'),
(402,'Business Loan','Business support',50000,500000,12,60,12.5,'Flat',true,true,'Active'),
(403,'Emergency Loan','Short term',1000,20000,1,6,18,'Flat',false,false,'Active'),
(404,'Education Loan','School fees',2000,50000,6,24,10,'Reducing',true,false,'Active'),
(405,'Agriculture Loan','Farm support',3000,150000,6,48,11,'Reducing',true,true,'Active');

-- SAVING ACCOUNT
INSERT INTO SavingAccount
(account_id,member_id,account_number,branch_id,product_id,current_balance,open_date,status) VALUES
(501,201,'SA1001',1,301,25000,'2021-01-10','Active'),
(502,202,'SA1002',1,301,12500,'2021-02-20','Active'),
(503,203,'SA1003',2,302,8000,'2021-06-15','Active'),
(504,204,'SA1004',2,303,50000,'2022-01-05','Active'),
(505,205,'SA1005',3,304,15000,'2022-03-20','Active');

-- LOAN
INSERT INTO Loan
```

```
(loan_id,member_id,product_id,loan_officer_id,principal_amount,interest_rate,term_months,disbursement_date,m
aturity_date,outstanding_principal,outstanding_interest,outstanding_fees,status) VALUES
(601,201,401,102,50000,15,24,'2023-01-15','2025-01-15',30000,4500,0,'Active'),
(602,203,403,103,10000,18,3,'2024-06-01','2024-09-01',0,0,0,'Closed'),
(603,204,402,104,200000,12.5,36,'2024-03-10','2027-03-10',170000,21250,500,'Active'),
(604,205,401,105,30000,15,12,NULL,NULL,30000,0,0,'Approved'),
(605,202,404,101,20000,10,18,'2024-01-10','2025-07-10',12000,1800,0,'Active');

-- LOAN SCHEDULE
INSERT INTO LoanSchedule
(schedule_id,loan_id,installment_number,due_date,principal_due,interest_due,fees_due,principal_paid,interest
_paid,fees_paid,status) VALUES
(701,601,1,'2023-02-15',2083,625,0,2083,625,0,'Paid'),
(702,601,2,'2023-03-15',2083,598,0,2083,598,0,'Paid'),
(703,601,3,'2023-04-15',2083,572,0,0,0,0,'Overdue'),
(704,603,1,'2024-04-10',5555,2083,0,5555,2083,0,'Paid'),
(705,605,1,'2024-02-10',1111,166,0,1111,166,0,'Paid');

-- SAVINGS TRANSACTION
INSERT INTO SavingsTransaction
(transaction_id,account_id,transaction_type,amount,transaction_date,balance_after_transaction,reference_numb
er,employee_id) VALUES
(801,501,'Deposit',10000,'2021-01-10 09:00:00',10000,'STX1',103),
(802,501,'Deposit',15000,'2021-06-15 10:30:00',25000,'STX2',103),
(803,502,'Deposit',20000,'2021-02-20 11:00:00',20000,'STX3',103),
(804,502,'Withdrawal',7500,'2022-08-10 14:15:00',12500,'STX4',103),
(805,503,'Deposit',8000,'2021-06-15 09:45:00',8000,'STX5',103);

      -- LOAN TRANSACTION
INSERT INTO LoanTransaction
(transaction_id,loan_id,loan_schedule_id,transaction_type,amount,transaction_date,payment_method,reference_n
umber,employee_id,reversal_status) VALUES
(901,601,NULL,'Disbursement',50000,'2023-01-15 08:00:00','Transfer','LTX1',102,'None'),
(902,601,701,'Repayment',2708,'2023-02-15 09:30:00','Cash','LTX2',103,'None'),
(903,603,NULL,'Disbursement',200000,'2024-03-10 08:30:00','Transfer','LTX3',104,'None'),
(904,603,704,'Repayment',7638,'2024-04-10 11:00:00','Transfer','LTX4',103,'None'),
(905,605,NULL,'Disbursement',20000,'2024-01-10 10:00:00','Transfer','LTX5',101,'None');

-- GUARANTY
INSERT INTO Guaranty (loan_id,member_id,guarantee_amount,status) VALUES
(601,202,25000,'Active'),
(603,203,100000,'Active'),
(604,201,15000,'Active'),
(605,204,10000,'Active');

-- COLLATERAL
INSERT INTO Collateral
(collateral_id,loan_id,collateral_type,description,estimated_value,ownership_document_ref,status) VALUES
(1101,603,'Vehicle','Toyota Vitz',350000,'DOC1','Pledged'),
(1102,603,'Property','Land 200sqm',500000,'DOC2','Pledged'),
(1103,605,'House','Small house',300000,'DOC3','Pledged'),
(1104,602,'Equipment','Office equipment',50000,'DOC4','Released');

--AUDIT
INSERT INTO Audit (audit_id, entity_name, entity_id, action_type, old_values, new_values, employee_id,
created_at, ip_address) VALUES
```

```
(1201,'Loan','601','Create',NULL,'{"status":"Applied"}',102,'2023-01-15 08:00:00','192.168.1.10'),
(1202,'Loan','601','Update','{"status":"Applied"}','{"status":"Disbursed"}',101,'2023-01-15
08:05:00','192.168.1.11'),
(1203,'Member','201','Create',NULL,'{"status":"Created"}',102,'2021-01-10 09:00:00','192.168.1.10'),
(1204,'Loan','603','Update','{"status":"Approved"}','{"status":"Active"}',104,'2024-03-10
09:00:00','192.168.1.12');

-- FEE TYPE
INSERT INTO FeeType (fee_type_id,name,description,calculation_method,amount_or_rate,is_active) VALUES
(1301,'Processing Fee','Loan fee','Percentage',2,true),
(1302,'Late Fee','Penalty','Percentage',5,true),
(1303,'Service Fee','Monthly fee','Flat',25,true),
(1304,'Registration Fee','Joining fee','Flat',100,true);

-- FEE EVENT
INSERT INTO FeeEvent (fee_event_id,fee_type_id,loan_id,saving_transaction_id,paid) VALUES
(1401,1301,601,NULL,true),
(1402,1302,601,NULL,false),
(1403,1301,603,NULL,true),
(1404,1304,605,NULL,true);

-- FEE TRANSACTION
INSERT INTO FeeTransaction (fee_transaction_id,fee_event_id,amount,reference,transaction_date) VALUES
(1501,1401,1000,'FTX1','2023-01-15 08:10:00'),
(1502,1403,4000,'FTX2','2024-03-10 08:40:00'),
(1503,1404,500,'FTX3','2024-01-10 08:20:00');
```

These are simple SQL examples (SELECT, UPDATE, ALTER, DELETE) based directly on our SACCO schema.

SELECT (Retrieve Data)

1. **Example:** `SELECT * FROM Member;`

2. Get members from a specific branch

Example :

```
SELECT * FROM Member
WHERE branch_id = 1;
```

3. Update member phone number

Example:

```
UPDATE Member
SET phone_number =
'0912345678'
WHERE member_id = 201;
```

4. Update savings account balance

```
UPDATE SavingAccount
SET current_balance =
current_balance + 5000
WHERE account_id = 501;
```

5. Delete a member

```
DELETE FROM Member
WHERE member_id= 002;
```

6. Add a new column to a member table

```
ALTER TABLE Member
ADD marital_status
VARCHAR(20);
```

3.2 Sample Outputs of the Inserted Values (Part of Annex)

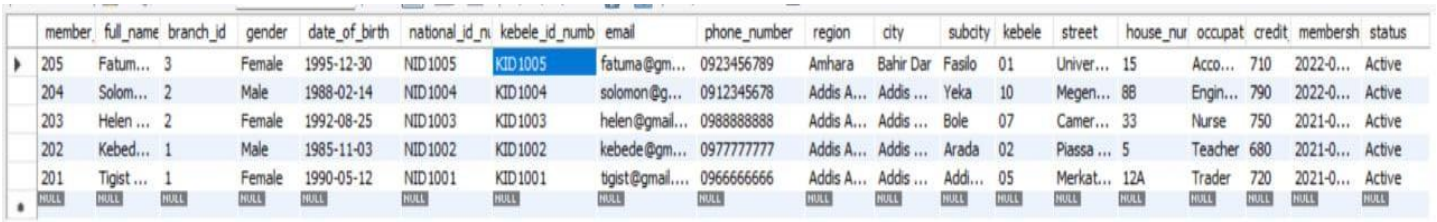
Branch table

branch_id	name	region	city	subcity	kebele	street	phone_number	manager_id	created_date	status
1	Merkato Branch	Addis Ababa	Addis Ababa	Addis Ketema	05	Merkato Ave	0111234567	NULL	2020-01-15	Active
2	Bole Branch	Addis Ababa	Addis Ababa	Bole	03	Bole Rd	0119876543	NULL	2020-06-01	Active
3	Bahir Dar Branch	Amhara	Bahir Dar	Fasilo	01	Lake Tana St	0581234567	NULL	2021-03-10	Active
4	Jimma Branch	Oromia	Jimma	Jimma	02	Main St	0471112233	NULL	2021-08-12	Active
5	Hawassa Branch	SNNPR	Hawassa	Tabor	04	Unity St	0462223344	NULL	2022-02-10	Active
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Employee table

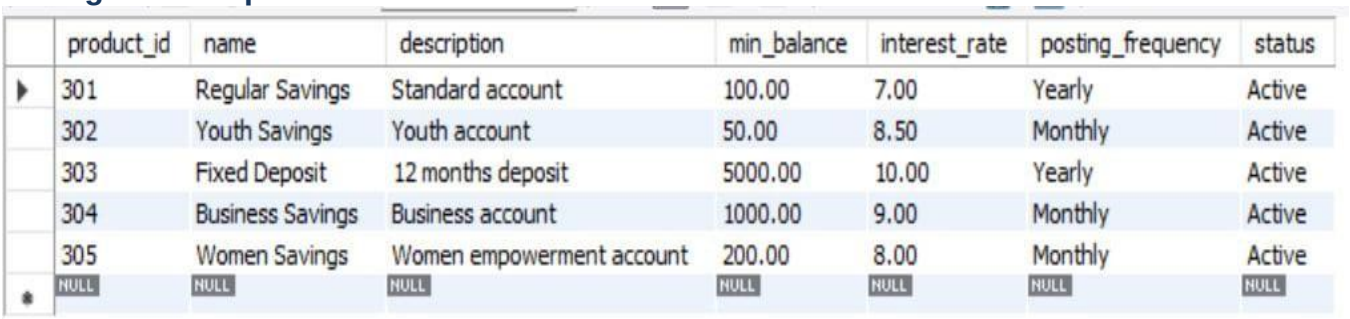
employee_id	branch_id	full_name	email	phone_number	role	hire_date	status
101	1	Abebe Kebede	abebe@yisakal.com	0911111111	Manager	2020-01-20	Active
102	1	Sara Tesfaye	sara@yisakal.com	0922222222	Officer	2020-03-15	Active
103	2	Dawit Haile	dawit@yisakal.com	0933333333	Teller	2021-01-10	Active
104	2	Meron Alemu	meron@yisakal.com	0944444444	Manager	2020-06-05	Active
105	3	Yonas Girma	yonas@yisakal.com	0955555555	Officer	2021-04-01	Active
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Member table



member	full_name	branch_id	gender	date_of_birth	national_id_n	kebele_id_numb	email	phone_number	region	city	subcity	kebele	street	house_nur	occupat	credit	membersh	status
205	Fatum...	3	Female	1995-12-30	NID1005	KID1005	fatuma@gm...	0923456789	Amhara	Bahir Dar	Fasilo	01	Univer...	15	Acco...	710	2022-0...	Active
204	Solom...	2	Male	1988-02-14	NID1004	KID1004	solomon@g...	0912345678	Addis A...	Addis ...	Yeka	10	Megen...	88	Engin...	790	2022-0...	Active
203	Helen ...	2	Female	1992-08-25	NID1003	KID1003	helen@gmail...	0988888888	Addis A...	Addis ...	Bole	07	Camer...	33	Nurse	750	2021-0...	Active
202	Kebed...	1	Male	1985-11-03	NID1002	KID1002	kebede@gm...	0977777777	Addis A...	Addis ...	Arada	02	Piassa ...	5	Teacher	680	2021-0...	Active
201	Tigist ...	1	Female	1990-05-12	NID1001	KID1001	tigist@gmail....	0966666666	Addis A...	Addis ...	Addi...	05	Merkat...	12A	Trader	720	2021-0...	Active
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

saving account product table

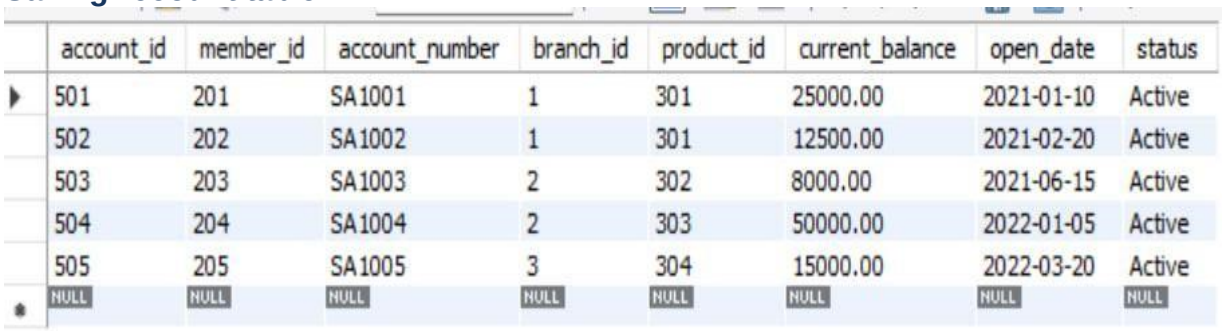


product_id	name	description	min_balance	interest_rate	posting_frequency	status
301	Regular Savings	Standard account	100.00	7.00	Yearly	Active
302	Youth Savings	Youth account	50.00	8.50	Monthly	Active
303	Fixed Deposit	12 months deposit	5000.00	10.00	Yearly	Active
304	Business Savings	Business account	1000.00	9.00	Monthly	Active
305	Women Savings	Women empowerment account	200.00	8.00	Monthly	Active
NULL	NULL	NULL	NULL	NULL	NULL	NULL

LoanProduct

product_id	name	description	min_amount	max_amount	min_tenure_months	max_tenure_months	base_interest_rate	interest_type	requires_guarantor	requires_collateral	status
401	Personal Loan	Personal needs	5000.00	100000.00	6	36	15.00	Reducing	1	0	Active
402	Business Loan	Business support	50000.00	500000.00	12	60	12.50	Flat	1	1	Active
403	Emergency Loan	Short term	1000.00	20000.00	1	6	18.00	Flat	0	0	Active
404	Education Loan	School fees	2000.00	50000.00	6	24	10.00	Reducing	1	0	Active
405	Agriculture Loan	Farm support	3000.00	150000.00	6	48	11.00	Reducing	1	1	Active
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SavingAccount table



account_id	member_id	account_number	branch_id	product_id	current_balance	open_date	status
501	201	SA1001	1	301	25000.00	2021-01-10	Active
502	202	SA1002	1	301	12500.00	2021-02-20	Active
503	203	SA1003	2	302	8000.00	2021-06-15	Active
504	204	SA1004	2	303	50000.00	2022-01-05	Active
505	205	SA1005	3	304	15000.00	2022-03-20	Active
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Loan table

account_id	member_id	account_number	branch_id	product_id	current_balance	open_date	status
501	201	SA1001	1	301	25000.00	2021-01-10	Active
502	202	SA1002	1	301	12500.00	2021-02-20	Active
503	203	SA1003	2	302	8000.00	2021-06-15	Active
504	204	SA1004	2	303	50000.00	2022-01-05	Active
505	205	SA1005	3	304	15000.00	2022-03-20	Active
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

loanSchedule table

schedule_id	loan_id	installment_number	due_date	principal_due	interest_due	fees_due	principal_paid	interest_paid	fees_paid	status
701	601	1	2023-02-15	2083.00	625.00	0.00	2083.00	625.00	0.00	Paid
702	601	2	2023-03-15	2083.00	598.00	0.00	2083.00	598.00	0.00	Paid
703	601	3	2023-04-15	2083.00	572.00	0.00	0.00	0.00	0.00	Overdue
704	603	1	2024-04-10	5555.00	2083.00	0.00	5555.00	2083.00	0.00	Paid
705	605	1	2024-02-10	1111.00	166.00	0.00	1111.00	166.00	0.00	Paid

LoanTransaction table

transaction_id	account_id	transaction_type	amount	transaction_date	balance_after_transaction	reference_number	employee_id
801	501	Deposit	10000.00	2021-01-10 09:00:00	10000.00	STX1	103
802	501	Deposit	15000.00	2021-06-15 10:30:00	25000.00	STX2	103
803	502	Deposit	20000.00	2021-02-20 11:00:00	20000.00	STX3	103
804	502	Withdrawal	7500.00	2022-08-10 14:15:00	12500.00	STX4	103
805	503	Deposit	8000.00	2021-06-15 09:45:00	8000.00	STX5	103

Guaranty

loan_id	member_id	guarantee_amount	status
601	202	25000.00	Active
603	203	100000.00	Active
604	201	15000.00	Active
605	204	10000.00	Active
NULL	NULL	NULL	NULL

Collateral table

collateral_id	loan_id	collateral_type	description	estimated_value	ownership_document_ref	status
1101	603	Vehicle	Toyota Vitz	350000.00	DOC1	Pledged
1102	603	Property	Land 200sqm	500000.00	DOC2	Pledged
1103	605	House	Small house	300000.00	DOC3	Pledged
1104	602	Equipment	Office equipment	50000.00	DOC4	Released
NULL	NULL	NULL	NULL	NULL	NULL	NULL

FeeType table

fee_type_id	name	description	calculation_method	amount_or_rate	is_active
1301	Processing Fee	Loan fee	Percentage	2.00	1
1302	Late Fee	Penalty	Percentage	5.00	1
1303	Service Fee	Monthly fee	Flat	25.00	1
1304	Registration Fee	Joining fee	Flat	100.00	1
NULL	NULL	NULL	NULL	NULL	NULL

FeeEventtable

fee_event_id	fee_type_id	loan_id	saving_transaction_id	paid
1401	1301	601	NULL	1
1402	1302	601	NULL	0
1403	1301	603	NULL	1
1404	1304	605	NULL	1
NULL	NULL	NULL	NULL	NULL

FeeTransaction table

fee_transaction_id	fee_event_id	amount	reference	transaction_date
1501	1401	1000.00	FTX1	2023-01-15 08:10:00
1502	1403	4000.00	FTX2	2024-03-10 08:40:00
1503	1404	500.00	FTX3	2024-01-10 08:20:00
NULL	NULL	NULL	NULL	NULL

3.2 MongoDB Implementation for Yisakal SACCO

The Yisakal_SACCO system is implemented using MongoDB as the NoSQL database solution alongside MySQL. MongoDB stores and manages semi-structured data using collections and documents.

The database contains collections such as Branch, Employee, Member, SavingAccount, Loan, LoanTransaction, SavingsTransaction, Guaranty, Collateral, Audit, FeeType, FeeEvent, and FeeTransaction.

Embedded documents are used for complex attributes like addresses, contact information, identities, balances, and payment details. References between collections are implemented using document IDs.

Sample data represents real SACCO operations including member registration, account management, loan processing, transaction handling, fee management, collateral tracking, and auditing activities.

The system is implemented using:

- MongoDB Database
- Mongosh (MongoDB Shell)
- JavaScript-based database scripts

The implementation is divided into:

- Database creation
- Collection creation with schema validation
- Indexing for performance
- Data insertion (seed data)
- Query verification

1. Database Creation

The implementation starts by creating and selecting the database:

```
use("Yisakal_SACCO");
```

2. Collections Implementation

The system is implemented using multiple collections. Each collection represents a real-world SACCO entity.

Core Collections

- Branch
- Employee
- Member
- LoanProduct
- SavingAccount
- Loan

Transaction Collections

- SavingsTransaction
- LoanTransaction
- FeeTransaction
- FeeEvent

Support Collections

- Guaranty
- Collateral
- Audit
- FeeType

Example:

- Employee → Branch (branch_id)

- Loan → Member (member_id)
- SavingAccount → Member (member_id)
- LoanTransaction → Loan (loan_id)
- Guaranty → Member and Loan (member_id, loan_id)

Schema Validation

Each collection uses **MongoDB JSON Schema Validator** to enforce data rules.

Example:

- Required fields (e.g., name, full_name)
- Data types (string, double, int, date)
- Enum constraints (e.g., Active/Inactive, Loan status types)

Indexing Strategy

Examples:

```
db.Employee.createIndex({ branch_id: 1 });

db.Member.createIndex({ branch_id: 1 });

db.SavingAccount.createIndex({ account_number: 1 }, { unique: true });

db.LoanSchedule.createIndex({ loan_id: 1, installment_number: 1 }, { unique:
true });
```

This is the full MongoDB implementation for the Yisakal SACCO system, which includes database creation, collection definitions, schema validation, indexing, and sample data insertion to support all SACCO operations.

```
// MongoDB Schema Validation & Initialization for Yisakal_SACCO database
 // run with mongosh < init_db.js

 use("Yisakal_SACCO");

 // 1. Branch
 db.createCollection("Branch", {
```

```
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name"],
      properties: {
        _id: { bsonType: "string" },
        name: { bsonType: "string", maxLength: 100 },
        phone_number: { bsonType: "string", maxLength: 50 },
        manager_id: { bsonType: "string", description: "ref: Employee" },
        created_date: { bsonType: "date" },
        status: { bsonType: "string", enum: ["Active", "Inactive"] },
        address: {
          bsonType: "object",
          properties: {
            region: { bsonType: "string", maxLength: 100 },
            city: { bsonType: "string", maxLength: 100 },
            subcity: { bsonType: "string", maxLength: 100 },
            kebele: { bsonType: "string", maxLength: 100 },
            street: { bsonType: "string", maxLength: 150 }
          }
        }
      }
    }
  }
});


// 2. Employee
db.createCollection("Employee", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["full_name", "role"],
      properties: {
        _id: { bsonType: "string" },
        branch_id: { bsonType: "string", description: "ref: Branch" },
        full_name: { bsonType: "string", maxLength: 150 },
        email: { bsonType: "string", maxLength: 150 },
        phone_number: { bsonType: "string", maxLength: 50 },
        role: { bsonType: "string", enum: ["Admin", "Manager", "Officer", "Teller"] },
        hire_date: { bsonType: "date" },
        status: { bsonType: "string", enum: ["Active", "Inactive", "Terminated"] }
      }
    }
```

```
  }
});

db.Employee.createIndex({ branch_id: 1 });


// 3. Member
db.createCollection("Member", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["full_name"],
      properties: {
        _id: { bsonType: "string" },
        full_name: { bsonType: "string", maxLength: 150 },
        branch_id: { bsonType: "string", description: "ref: Branch" },
        gender: { bsonType: "string", enum: ["Male", "Female"] },
        date_of_birth: { bsonType: "date" },
        occupation: { bsonType: "string", maxLength: 100 },
        credit_score: { bsonType: "int" },
        membership_date: { bsonType: "date" },
        status: { bsonType: "string", enum: ["Active", "Inactive", "Suspended"] },
        identity: {
          bsonType: "object",
          description: "Government-issued identification documents",
          properties: {
            national_id_number: { bsonType: "string", maxLength: 50 },
            kebele_id_number: { bsonType: "string", maxLength: 50 }
          }
        },
        contact: {
          bsonType: "object",
          properties: {
            email: { bsonType: "string", maxLength: 150 },
            phone_number: { bsonType: "string", maxLength: 50 }
          }
        },
        address: {
          bsonType: "object",
          properties: {
            region: { bsonType: "string", maxLength: 100 },
            city: { bsonType: "string", maxLength: 100 },
            subcity: { bsonType: "string", maxLength: 100 },
            kebele: { bsonType: "string", maxLength: 100 },
```

```
            street: { bsonType: "string", maxLength: 150 },
            house_number: { bsonType: "string", maxLength: 50 }
          }
        }
      }
    }
  }
});

db.Member.createIndex({ branch_id: 1 });
db.Member.createIndex({ "identity.national_id_number": 1 }, { unique: true, sparse: true
});
db.Member.createIndex({ "identity.kebele_id_number": 1 }, { unique: true, sparse: true
});


// 4. SavingAccountProduct
db.createCollection("SavingAccountProduct", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "interest_rate", "posting_frequency"],
      properties: {
        _id: { bsonType: "string" },
        name: { bsonType: "string", maxLength: 100 },
        description: { bsonType: "string", maxLength: 255 },
        min_balance: { bsonType: "double" },
        interest_rate: { bsonType: "double" },
        posting_frequency: { bsonType: "string", enum: ["Daily", "Weekly", "Monthly",
"Yearly"] },
        status: { bsonType: "string", enum: ["Active", "Inactive"] }
      }
    }
  }
});


// 5. LoanProduct
db.createCollection("LoanProduct", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "interest_type"],
      properties: {
```

```
        _id: { bsonType: "string" },
        name: { bsonType: "string", maxLength: 100 },
        description: { bsonType: "string", maxLength: 255 },
        base_interest_rate: { bsonType: "double" },
        interest_type: { bsonType: "string", enum: ["Flat", "Reducing"] },
        requires_guarantor: { bsonType: "bool" },
        requires_collateral: { bsonType: "bool" },
        status: { bsonType: "string", enum: ["Active", "Inactive"] },
        amount: {
          bsonType: "object",
          description: "Eligible loan amount range",
          properties: {
            min: { bsonType: "double" },
            max: { bsonType: "double" }
          }
        },
        tenure: {
          bsonType: "object",
          description: "Loan tenure range in months",
          properties: {
            min_months: { bsonType: "int" },
            max_months: { bsonType: "int" }
          }
        }
      }
    }
  }
});


// 6. SavingAccount
db.createCollection("SavingAccount", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["member_id", "account_number", "product_id"],
      properties: {
        _id: { bsonType: "string" },
        member_id: { bsonType: "string", description: "ref: Member" },
        account_number: { bsonType: "string", maxLength: 50 },
        branch_id: { bsonType: "string", description: "ref: Branch" },
        product_id: { bsonType: "string", description: "ref: SavingAccountProduct" },
        current_balance: { bsonType: "double" },
        open_date: { bsonType: "date" },
```

```
        status: { bsonType: "string", enum: ["Active", "Inactive", "Closed"] }
      }
    }
  }
});

db.SavingAccount.createIndex({ account_number: 1 }, { unique: true });
db.SavingAccount.createIndex({ member_id: 1 });
db.SavingAccount.createIndex({ product_id: 1 });
db.SavingAccount.createIndex({ branch_id: 1 });


// 7. Loan
db.createCollection("Loan", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["member_id", "product_id", "principal_amount", "interest_rate",
"term_months"],
      properties: {
        _id: { bsonType: "string" },
        member_id: { bsonType: "string", description: "ref: Member" },
        product_id: { bsonType: "string", description: "ref: LoanProduct" },
        loan_officer_id: { bsonType: "string", description: "ref: Employee" },
        principal_amount: { bsonType: "double" },
        interest_rate: { bsonType: "double" },
        term_months: { bsonType: "int" },
        disbursement_date: { bsonType: "date" },
        maturity_date: { bsonType: "date" },
        status: { bsonType: "string", enum: ["Applied", "Approved", "Disbursed",
"Active", "Closed", "Defaulted"] },
        outstanding: {
          bsonType: "object",
          description: "Running balances of amounts still owed",
          properties: {
            principal: { bsonType: "double" },
            interest: { bsonType: "double" },
            fees: { bsonType: "double" }
          }
        }
      }
    }
  }
});
```

```
db.Loan.createIndex({ member_id: 1 });
db.Loan.createIndex({ product_id: 1 });
db.Loan.createIndex({ loan_officer_id: 1 });
db.Loan.createIndex({ status: 1 });


// 8. LoanSchedule
db.createCollection("LoanSchedule", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["loan_id", "installment_number", "due_date"],
      properties: {
        _id: { bsonType: "string" },
        loan_id: { bsonType: "string", description: "ref: Loan" },
        installment_number: { bsonType: "int" },
        due_date: { bsonType: "date" },
        status: { bsonType: "string", enum: ["Pending", "Paid", "Partial", "Overdue"] },
        due: {
          bsonType: "object",
          description: "Amounts due on this installment",
          properties: {
            principal: { bsonType: "double" },
            interest: { bsonType: "double" },
            fees: { bsonType: "double" }
          }
        },
        paid: {
          bsonType: "object",
          description: "Amounts already paid on this installment",
          properties: {
            principal: { bsonType: "double" },
            interest: { bsonType: "double" },
            fees: { bsonType: "double" }
          }
        }
      }
    }
  }
});

db.LoanSchedule.createIndex({ loan_id: 1 });
db.LoanSchedule.createIndex({ loan_id: 1, installment_number: 1 }, { unique: true });
```

```
db.LoanSchedule.createIndex({ due_date: 1, status: 1 });


// 9. SavingsTransaction
db.createCollection("SavingsTransaction", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["account_id", "transaction_type", "amount"],
      properties: {
        _id: { bsonType: "string" },
        account_id: { bsonType: "string", description: "ref: SavingAccount" },
        employee_id: { bsonType: "string", description: "ref: Employee" },
        transaction_type: { bsonType: "string", enum: ["Deposit", "Withdrawal",
"Interest", "Fee"] },
        amount: { bsonType: "double" },
        balance_after_transaction: { bsonType: "double" },
        transaction_date: { bsonType: "date" },
        reference_number: { bsonType: "string", maxLength: 100 }
      }
    }
  }
});

db.SavingsTransaction.createIndex({ account_id: 1 });
db.SavingsTransaction.createIndex({ employee_id: 1 });
db.SavingsTransaction.createIndex({ transaction_date: -1 });


// 10. LoanTransaction
db.createCollection("LoanTransaction", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["loan_id", "transaction_type", "amount", "payment_method"],
      properties: {
        _id: { bsonType: "string" },
        loan_id: { bsonType: "string", description: "ref: Loan" },
        loan_schedule_id: { bsonType: "string", description: "ref: LoanSchedule (nullable
for disbursements)" },
        employee_id: { bsonType: "string", description: "ref: Employee" },
        transaction_type: { bsonType: "string", enum: ["Disbursement", "Repayment",
"Interest", "Fee"] },
        amount: { bsonType: "double" },
```

```
        transaction_date: { bsonType: "date" },
        payment_method: { bsonType: "string", enum: ["Cash", "Transfer", "Check"] },
        reference_number: { bsonType: "string", maxLength: 100 },
        reversal_status: { bsonType: "string", enum: ["None", "Reversed", "Correction"] }
      }
    }
  }
});

db.LoanTransaction.createIndex({ loan_id: 1 });
db.LoanTransaction.createIndex({ loan_schedule_id: 1 });
db.LoanTransaction.createIndex({ employee_id: 1 });
db.LoanTransaction.createIndex({ transaction_date: -1 });


// 11. Guaranty
db.createCollection("Guaranty", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["loan_id", "member_id", "guarantee_amount"],
      properties: {
        _id: { bsonType: "string" },
        loan_id: { bsonType: "string", description: "ref: Loan" },
        member_id: { bsonType: "string", description: "ref: Member (the guarantor)" },
        guarantee_amount: { bsonType: "double" },
        status: { bsonType: "string", enum: ["Active", "Released", "Claimed"] }
      }
    }
  }
});

db.Guaranty.createIndex({ loan_id: 1, member_id: 1 }, { unique: true });
db.Guaranty.createIndex({ member_id: 1 });

// 12. Collateral
db.createCollection("Collateral", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["loan_id", "collateral_type"],
      properties: {
        _id: { bsonType: "string" },
        loan_id: { bsonType: "string", description: "ref: Loan" },
```

```
        collateral_type: { bsonType: "string", maxLength: 100 },
        description: { bsonType: "string", maxLength: 255 },
        estimated_value: { bsonType: "double" },
        ownership_document_ref: { bsonType: "string", maxLength: 150 },
        status: { bsonType: "string", enum: ["Pledged", "Released", "Liquidated"] }
      }
    }
  }
});

db.Collateral.createIndex({ loan_id: 1 });


// 13. Audit
db.createCollection("Audit", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["entity_name", "entity_id", "action_type"],
      properties: {
        _id: { bsonType: "string" },
        entity_name: { bsonType: "string", enum: ["Member", "Loan", "SavingAccount",
"Employee", "Branch"] },
        entity_id: { bsonType: "string" },
        action_type: { bsonType: "string", enum: ["Create", "Update", "Delete", "Login"]
},
        employee_id: { bsonType: "string", description: "ref: Employee" },
        created_at: { bsonType: "date" },
        ip_address: { bsonType: "string", maxLength: 45 },
        changes: {
          bsonType: "object",
          description: "Snapshot of what changed",
          properties: {
            old_values: { bsonType: "object" },
            new_values: { bsonType: "object" }
          }
        }
      }
    }
  }
});

db.Audit.createIndex({ entity_name: 1, entity_id: 1 });
db.Audit.createIndex({ employee_id: 1 });
```

```
db.Audit.createIndex({ created_at: -1 });


// 14. FeeType
db.createCollection("FeeType", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "calculation_method", "amount_or_rate"],
      properties: {
        _id: { bsonType: "string" },
        name: { bsonType: "string", maxLength: 100 },
        description: { bsonType: "string", maxLength: 255 },
        calculation_method: { bsonType: "string", enum: ["Flat", "Percentage"] },
        amount_or_rate: { bsonType: "double" },
        is_active: { bsonType: "bool" }
      }
    }
  }
});

// 15. FeeEvent
db.createCollection("FeeEvent", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["fee_type_id"],
      properties: {
        _id: { bsonType: "string" },
        fee_type_id: { bsonType: "string", description: "ref: FeeType" },
        loan_id: { bsonType: "string", description: "ref: Loan (nullable)" },
        saving_transaction_id: { bsonType: "string", description: "ref:
SavingsTransaction (nullable)" },
        paid: { bsonType: "bool" }
      }
    }
  }
});

db.FeeEvent.createIndex({ fee_type_id: 1 });
db.FeeEvent.createIndex({ loan_id: 1 });
db.FeeEvent.createIndex({ saving_transaction_id: 1 });
```

```
// 16. FeeTransaction
db.createCollection("FeeTransaction", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["fee_event_id", "amount"],
      properties: {
        _id: { bsonType: "string" },
        fee_event_id: { bsonType: "string", description: "ref: FeeEvent" },
        amount: { bsonType: "double" },
        reference: { bsonType: "string", maxLength: 100 },
        transaction_date: { bsonType: "date" }
      }
    }
  }
});

db.FeeTransaction.createIndex({ fee_event_id: 1 });
db.FeeTransaction.createIndex({ transaction_date: -1 });

print("   Yisakal_SACCO MongoDB schema initialized successfully.");
print(`   Collections created: ${db.getCollectionNames().length}`);
```

Sample Data Implementation (Seed Data)

The system is populated using insertMany() operations.

Example:

Branch Data: This represents real SACCO branch operations.

```
db.Branch.insertMany([
  {
    _id: "Branch-1",
    name: "Merkato Branch",
    address: { region: "Addis Ababa", city: "Addis Ababa", subcity: "Addis
Ketema" },
    phone_number: "0111234567",
    status: "Active"
  }
]);
```

Member data: stores member registration information.

```
db.Member.insertMany([
  {
    _id: "Member-201",
    full_name: "Tigist Bekele",
    identity: {
      national_id_number: "ETH-NID-1001"
    },
    contact: {
      email: "tigist@gmail.com"
    }
  }
]);
```

Transaction Implementation

Example:

1. Savings Transaction: This records savings deposits and withdrawals.

```
db.SavingsTransaction.insertMany([
  {
    account_id: "SavingAccount-501",
    transaction_type: "Deposit",
    amount: 10000,
    balance_after_transaction: 25000
  }
]);
```

2. Loan Transaction Example: This tracks loan repayment activities.

```
db.LoanTransaction.insertMany([
{
loan_id: "Loan-601",
transaction_type: "Repayment",
amount: 2708.33,
payment_method: "Cash"
}
]);
```

Fee Management Implementation

Fee system includes:

- FeeType (definition of fees)
- FeeEvent (when fee is triggered)

- FeeTransaction (actual payment records)

3. The Audit collection tracks system activities: ensures transparency and accountability.

```
db.Audit.insertMany([
  {
    entity_name: "Loan",
    entity_id: "Loan-601",
    action_type: "Create",
    created_at: new Date()
  }
]);
```

Below the MongoDB seed data is included to populate the collections with sample records for testing, demonstration, and project presentation purposes. The data represents branches, employees, members, accounts, loans, transactions, fees, collateral, and audit activities within the SACCO management system.

```
// MongoDB Seed Data For Yisakal_SACCO
// Run after mongo_schema.js (mongosh Yisakal_SACCO --file mongo_seed.js)

use("Yisakal_SACCO");

// Branches
db.Branch.insertMany([

  { _id: "Branch-1", name: "Merkato Branch", address: { region: "Addis Ababa", city: "Addis
Ababa", subcity: "Addis Ketema", kebele: "05", street: "Merkato Ave" }, phone_number:
"0111234567", created_date: new Date("2020-01-15"), status: "Active" },
  { _id: "Branch-2", name: "Bole Branch", address: { region: "Addis Ababa", city: "Addis Ababa",
subcity: "Bole", kebele: "03", street: "Bole Rd" }, phone_number: "0119876543", created_date: new
Date("2020-06-01"), status: "Active" },
  { _id: "Branch-3", name: "Bahir Dar Branch", address: { region: "Amhara", city: "Bahir Dar",
subcity: "Fasilo", kebele: "01", street: "Lake Tana St" }, phone_number: "0581234567",
created_date: new Date("2021-03-10"), status: "Active" },
])

// Employees
db.Employee.insertMany([
  { _id: "Employee-101", branch_id: "Branch-1", full_name: "Abebe Kebede", email:
"abebe@yisakal.com", phone_number: "0911111111", role: "Manager", hire_date: new Date("2020-01-
20"), status: "Active" },
  { _id: "Employee-102", branch_id: "Branch-1", full_name: "Sara Tesfaye", email:
"sara@yisakal.com", phone_number: "0922222222", role: "Officer", hire_date: new Date("2020-03-
15"), status: "Active" },
  { _id: "Employee-103", branch_id: "Branch-2", full_name: "Dawit Haile", email:
"dawit@yisakal.com", phone_number: "0933333333", role: "Teller", hire_date: new Date("2021-01-
10"), status: "Active" },
```

```
  { _id: "Employee-104", branch_id: "Branch-2", full_name: "Meron Alemu", email:
"meron@yisakal.com", phone_number: "0944444444", role: "Manager", hire_date: new Date("2020-06-
05"), status: "Active" },
  { _id: "Employee-105", branch_id: "Branch-3", full_name: "Yonas Girma", email:
"yonas@yisakal.com", phone_number: "0955555555", role: "Officer", hire_date: new Date("2021-04-
01"), status: "Active" },
]);

// Link managers to branches
db.Branch.updateOne({ _id: "Branch-1" }, { $set: { manager_id: "Employee-101" } });
db.Branch.updateOne({ _id: "Branch-2" }, { $set: { manager_id: "Employee-104" } });
db.Branch.updateOne({ _id: "Branch-3" }, { $set: { manager_id: "Employee-105" } });

// Members
db.Member.insertMany([
  { _id: "Member-201", full_name: "Tigist Bekele", branch_id: "Branch-1", gender: "Female",
date_of_birth: new Date("1990-05-12"), identity: { national_id_number: "ETH-NID-1001",
kebele_id_number: "KEB-1001" }, contact: { email: "tigist@gmail.com", phone_number: "0966666666"
}, address: { region: "Addis Ababa", city: "Addis Ababa", subcity: "Addis Ketema", kebele: "05",

street: "Merkato Ave", house_number: "12A" }, occupation: "Trader", credit_score: 720,
membership_date: new Date("2021-01-10"), status: "Active" },
  { _id: "Member-202", full_name: "Kebede Desta", branch_id: "Branch-1", gender: "Male",
date_of_birth: new Date("1985-11-03"), identity: { national_id_number: "ETH-NID-1002",
kebele_id_number: "KEB-1002" }, contact: { email: "kebede@gmail.com", phone_number: "0977777777"
}, address: { region: "Addis Ababa", city: "Addis Ababa", subcity: "Arada", kebele: "02", street:
"Piassa Rd", house_number: "5" }, occupation: "Teacher", credit_score: 680, membership_date: new
Date("2021-02-20"), status: "Active" },
  { _id: "Member-203", full_name: "Helen Tadesse", branch_id: "Branch-2", gender: "Female",
date_of_birth: new Date("1992-08-25"), identity: { national_id_number: "ETH-NID-1003",
kebele_id_number: "KEB-1003" }, contact: { email: "helen@gmail.com", phone_number: "0988888888" },
address: { region: "Addis Ababa", city: "Addis Ababa", subcity: "Bole", kebele: "07", street:
"Cameroon St", house_number: "33" }, occupation: "Nurse", credit_score: 750, membership_date: new
Date("2021-06-15"), status: "Active" },
  { _id: "Member-204", full_name: "Solomon Gebre", branch_id: "Branch-2", gender: "Male",
date_of_birth: new Date("1988-02-14"), identity: { national_id_number: "ETH-NID-1004",
kebele_id_number: "KEB-1004" }, contact: { email: "solomon@gmail.com", phone_number: "0912345678"
}, address: { region: "Addis Ababa", city: "Addis Ababa", subcity: "Yeka", kebele: "10", street:
"Megenagna Rd", house_number: "8B" }, occupation: "Engineer", credit_score: 790, membership_date:
new Date("2022-01-05"), status: "Active" },
  { _id: "Member-205", full_name: "Fatuma Ahmed", branch_id: "Branch-3", gender: "Female",
date_of_birth: new Date("1995-12-30"), identity: { national_id_number: "ETH-NID-1005",
kebele_id_number: "KEB-1005" }, contact: { email: "fatuma@gmail.com", phone_number: "0923456789"
}, address: { region: "Amhara", city: "Bahir Dar", subcity: "Fasilo", kebele: "01", street:
"University Ave", house_number: "15" }, occupation: "Accountant", credit_score: 710,
membership_date: new Date("2022-03-20"), status: "Active" },
]);

// SavingAccountProduct

db.SavingAccountProduct.insertMany([
  { _id: "SavingAccountProduct-301", name: "Regular Savings", description: "Standard savings
account", min_balance: 100, interest_rate: 7.0, posting_frequency: "Yearly", status: "Active" },
  { _id: "SavingAccountProduct-302", name: "Youth Savings", description: "Savings for members
under 30", min_balance: 50, interest_rate: 8.5, posting_frequency: "Monthly", status: "Active" },
  { _id: "SavingAccountProduct-303", name: "Fixed Deposit", description: "12-month fixed deposit",
min_balance: 5000, interest_rate: 10.0, posting_frequency: "Yearly", status: "Active" },
```

```
]);

// LoanProduct
db.LoanProduct.insertMany([
  { _id: "LoanProduct-401", name: "Personal Loan", description: "General purpose personal loan", amount: { min: 5000, max: 100000 }, tenure: { min_months: 6, max_months: 36 }, base_interest_rate: 15.0, interest_type: "Reducing", requires_guarantor: true, requires_collateral: false, status: "Active" },
  { _id: "LoanProduct-402", name: "Business Loan", description: "Small business expansion loan", amount: { min: 50000, max: 500000 }, tenure: { min_months: 12, max_months: 60 }, base_interest_rate: 12.5, interest_type: "Flat", requires_guarantor: true, requires_collateral: true, status: "Active" },
  { _id: "LoanProduct-403", name: "Emergency Loan", description: "Short-term emergency loan", amount: { min: 1000, max: 20000 }, tenure: { min_months: 1, max_months: 6 }, base_interest_rate: 18.0, interest_type: "Flat", requires_guarantor: false, requires_collateral: false, status: "Active" },
]);

// SavingAccount
db.SavingAccount.insertMany([
  { _id: "SavingAccount-501", member_id: "Member-201", account_number: "SAV-2021-0001", branch_id: "Branch-1", product_id: "SavingAccountProduct-301", current_balance: 25000, open_date: new Date("2021-01-10"), status: "Active" },
  { _id: "SavingAccount-502", member_id: "Member-202", account_number: "SAV-2021-0002", branch_id:

"Branch-1", product_id: "SavingAccountProduct-301", current_balance: 12500, open_date: new Date("2021-02-20"), status: "Active" },
  { _id: "SavingAccount-503", member_id: "Member-203", account_number: "SAV-2021-0003", branch_id: "Branch-2", product_id: "SavingAccountProduct-302", current_balance: 8000, open_date: new Date("2021-06-15"), status: "Active" },
  { _id: "SavingAccount-504", member_id: "Member-204", account_number: "SAV-2022-0004", branch_id: "Branch-2", product_id: "SavingAccountProduct-303", current_balance: 50000, open_date: new Date("2022-01-05"), status: "Active" },
  { _id: "SavingAccount-505", member_id: "Member-205", account_number: "SAV-2022-0005", branch_id: "Branch-3", product_id: "SavingAccountProduct-301", current_balance: 15000, open_date: new Date("2022-03-20"), status: "Active" },
]);

// Loan
db.Loan.insertMany([
  { _id: "Loan-601", member_id: "Member-201", product_id: "LoanProduct-401", loan_officer_id: "Employee-102", principal_amount: 50000, interest_rate: 15.0, term_months: 24, disbursement_date: new Date("2023-01-15"), maturity_date: new Date("2025-01-15"), outstanding: { principal: 30000, interest: 4500, fees: 0 }, status: "Active" },
  { _id: "Loan-602", member_id: "Member-203", product_id: "LoanProduct-403", loan_officer_id: "Employee-103", principal_amount: 10000, interest_rate: 18.0, term_months: 3, disbursement_date: new Date("2024-06-01"), maturity_date: new Date("2024-09-01"), outstanding: { principal: 0, interest: 0, fees: 0 }, status: "Closed" },
  { _id: "Loan-603", member_id: "Member-204", product_id: "LoanProduct-402", loan_officer_id: "Employee-104", principal_amount: 200000, interest_rate: 12.5, term_months: 36, disbursement_date: new Date("2024-03-10"), maturity_date: new Date("2027-03-10"), outstanding: { principal: 170000, interest: 21250, fees: 500 }, status: "Active" },
  { _id: "Loan-604", member_id: "Member-205", product_id: "LoanProduct-401", loan_officer_id: "Employee-105", principal_amount: 30000, interest_rate: 15.0, term_months: 12, disbursement_date: null, maturity_date: null, outstanding: { principal: 30000, interest: 0, fees: 0 }, status: "Approved" },
]);
```

```
// LoanSchedule
db.LoanSchedule.insertMany([
  { _id: "LoanSchedule-701", loan_id: "Loan-601", installment_number: 1, due_date: new Date("2023-02-15"), due: { principal: 2083.33, interest: 625, fees: 0 }, paid: { principal: 2083.33, interest: 625, fees: 0 }, status: "Paid" },
  { _id: "LoanSchedule-702", loan_id: "Loan-601", installment_number: 2, due_date: new Date("2023-03-15"), due: { principal: 2083.33, interest: 598.96, fees: 0 }, paid: { principal: 2083.33, interest: 598.96, fees: 0 }, status: "Paid" },
  { _id: "LoanSchedule-703", loan_id: "Loan-601", installment_number: 3, due_date: new Date("2023-04-15"), due: { principal: 2083.33, interest: 572.92, fees: 0 }, paid: { principal: 0, interest: 0, fees: 0 }, status: "Overdue" },
  { _id: "LoanSchedule-704", loan_id: "Loan-603", installment_number: 1, due_date: new Date("2024-04-10"), due: { principal: 5555.56, interest: 2083.33, fees: 0 }, paid: { principal: 5555.56, interest: 2083.33, fees: 0 }, status: "Paid" },
  { _id: "LoanSchedule-705", loan_id: "Loan-603", installment_number: 2, due_date: new Date("2024-05-10"), due: { principal: 5555.56, interest: 2083.33, fees: 0 }, paid: { principal: 5555.56, interest: 2083.33, fees: 0 }, status: "Paid" },
]);

// SavingsTransaction
db.SavingsTransaction.insertMany([
  { _id: "SavingsTransaction-801", account_id: "SavingAccount-501", transaction_type: "Deposit", amount: 10000, balance_after_transaction: 10000, transaction_date: new Date("2021-01-10"), reference_number: "STX-0001", employee_id: "Employee-103" },
  { _id: "SavingsTransaction-802", account_id: "SavingAccount-501", transaction_type: "Deposit", amount: 15000, balance_after_transaction: 25000, transaction_date: new Date("2021-06-15"), reference_number: "STX-0002", employee_id: "Employee-103" },
  { _id: "SavingsTransaction-803", account_id: "SavingAccount-502", transaction_type: "Deposit",
```

```
amount: 20000, balance_after_transaction: 20000, transaction_date: new Date("2021-02-20"),
reference_number: "STX-0003", employee_id: "Employee-103" },
  { _id: "SavingsTransaction-804", account_id: "SavingAccount-502", transaction_type:
"Withdrawal", amount: 7500, balance_after_transaction: 12500, transaction_date: new Date("2022-08-
10"), reference_number: "STX-0004", employee_id: "Employee-103" },
  { _id: "SavingsTransaction-805", account_id: "SavingAccount-503", transaction_type: "Deposit",
amount: 8000, balance_after_transaction: 8000, transaction_date: new Date("2021-06-15"),
reference_number: "STX-0005", employee_id: "Employee-103" },
  { _id: "SavingsTransaction-806", account_id: "SavingAccount-504", transaction_type: "Deposit",
amount: 50000, balance_after_transaction: 50000, transaction_date: new Date("2022-01-05"),
reference_number: "STX-0006", employee_id: "Employee-103" },
]);

// LoanTransaction
db.LoanTransaction.insertMany([
  { _id: "LoanTransaction-901", loan_id: "Loan-601", loan_schedule_id: null, transaction_type:
"Disbursement", amount: 50000, transaction_date: new Date("2023-01-15"), payment_method:
"Transfer", reference_number: "LTX-0001", employee_id: "Employee-102", reversal_status: "None" },
  { _id: "LoanTransaction-902", loan_id: "Loan-601", loan_schedule_id: "LoanSchedule-701",
transaction_type: "Repayment", amount: 2708.33, transaction_date: new Date("2023-02-15"),
payment_method: "Cash", reference_number: "LTX-0002", employee_id: "Employee-103",
reversal_status: "None" },
  { _id: "LoanTransaction-903", loan_id: "Loan-601", loan_schedule_id: "LoanSchedule-702",
transaction_type: "Repayment", amount: 2682.29, transaction_date: new Date("2023-03-15"),
payment_method: "Cash", reference_number: "LTX-0003", employee_id: "Employee-103",
reversal_status: "None" },
  { _id: "LoanTransaction-904", loan_id: "Loan-603", loan_schedule_id: null, transaction_type:
"Disbursement", amount: 200000, transaction_date: new Date("2024-03-10"), payment_method:
"Transfer", reference_number: "LTX-0004", employee_id: "Employee-104", reversal_status: "None" },
  { _id: "LoanTransaction-905", loan_id: "Loan-603", loan_schedule_id: "LoanSchedule-704",
transaction_type: "Repayment", amount: 7638.89, transaction_date: new Date("2024-04-10"),
payment_method: "Transfer", reference_number: "LTX-0005", employee_id: "Employee-103",
reversal_status: "None" },
]);

// Guaranty
db.Guaranty.insertMany([
  { _id: "Guaranty-1001", loan_id: "Loan-601", member_id: "Member-202", guarantee_amount: 25000,
status: "Active" },
  { _id: "Guaranty-1002", loan_id: "Loan-603", member_id: "Member-203", guarantee_amount: 100000,
status: "Active" },
  { _id: "Guaranty-1003", loan_id: "Loan-604", member_id: "Member-201", guarantee_amount: 15000,
status: "Active" },
]);

// Collateral
db.Collateral.insertMany([
  { _id: "Collateral-1101", loan_id: "Loan-603", collateral_type: "Vehicle", description: "2018
Toyota Vitz, white", estimated_value: 350000, ownership_document_ref: "DOC-VEH-2024-001", status:
"Pledged" },
  { _id: "Collateral-1102", loan_id: "Loan-603", collateral_type: "Property Title", description:
"Land deed in Bole subcity, 200sqm", estimated_value: 500000, ownership_document_ref: "DOC-PROP-
2024-002", status: "Pledged" },
]);

// Audit
```

```
db.Audit.insertMany([
  { _id: "Audit-1201", entity_name: "Loan", entity_id: "Loan-601", action_type: "Create",
employee_id: "Employee-102", created_at: new Date("2023-01-15"), ip_address: "192.168.1.10",
changes: { old_values: null, new_values: { principal_amount: 50000, status: "Applied" } } },

  { _id: "Audit-1202", entity_name: "Loan", entity_id: "Loan-601", action_type: "Update",
employee_id: "Employee-101", created_at: new Date("2023-01-15"), ip_address: "192.168.1.11",
changes: { old_values: { status: "Applied" }, new_values: { status: "Disbursed" } } },
  { _id: "Audit-1203", entity_name: "Member", entity_id: "Member-201", action_type: "Create",
employee_id: "Employee-102", created_at: new Date("2021-01-10"), ip_address: "192.168.1.10",
changes: { old_values: null, new_values: { full_name: "Tigist Bekele" } } },
]);

// FeeType
db.FeeType.insertMany([
  { _id: "FeeType-1301", name: "Loan Processing Fee", description: "One-time fee on loan
approval", calculation_method: "Percentage", amount_or_rate: 2.0, is_active: true },
  { _id: "FeeType-1302", name: "Late Payment Penalty", description: "Charged on overdue
installments", calculation_method: "Percentage", amount_or_rate: 5.0, is_active: true },
  { _id: "FeeType-1303", name: "Account Maintenance Fee", description: "Monthly account
maintenance", calculation_method: "Flat", amount_or_rate: 25, is_active: true },
]);

// FeeEvent
db.FeeEvent.insertMany([
  { _id: "FeeEvent-1401", fee_type_id: "FeeType-1301", loan_id: "Loan-601", saving_transaction_id:
null, paid: true },
  { _id: "FeeEvent-1402", fee_type_id: "FeeType-1302", loan_id: "Loan-601", saving_transaction_id:
null, paid: false },
  { _id: "FeeEvent-1403", fee_type_id: "FeeType-1301", loan_id: "Loan-603", saving_transaction_id:
null, paid: true },
]);

// FeeTransaction
db.FeeTransaction.insertMany([
  { _id: "FeeTransaction-1501", fee_event_id: "FeeEvent-1401", amount: 1000, reference: "FTX-
0001", transaction_date: new Date("2023-01-15") },
  { _id: "FeeTransaction-1502", fee_event_id: "FeeEvent-1403", amount: 4000, reference: "FTX-
0002", transaction_date: new Date("2024-03-10") },
]);

print("   Seed data inserted successfully.");
print(`   Branches: ${db.Branch.countDocuments()}`);
print(`   Employees: ${db.Employee.countDocuments()}`);
print(`   Members: ${db.Member.countDocuments()}`);
print(`   SavingAccountProducts: ${db.SavingAccountProduct.countDocuments()}`);
print(`   LoanProducts: ${db.LoanProduct.countDocuments()}`);
print(`   SavingAccounts: ${db.SavingAccount.countDocuments()}`);
print(`   Loans: ${db.Loan.countDocuments()}`);
print(`   LoanSchedules: ${db.LoanSchedule.countDocuments()}`);
print(`   SavingsTransactions: ${db.SavingsTransaction.countDocuments()}`);
print(`   LoanTransactions: ${db.LoanTransaction.countDocuments()}`);
print(`   Guarantees: ${db.Guaranty.countDocuments()}`);
print(`   Collaterals: ${db.Collateral.countDocuments()}`);
```

```
print(`   Audits: ${db.Audit.countDocuments()}`);
print(`   FeeTypes: ${db.FeeType.countDocuments()}`);
print(`   FeeEvents: ${db.FeeEvent.countDocuments()}`);
print(`   FeeTransactions: ${db.FeeTransaction.countDocuments()}`);
```

Implementation Examples

The query implementation shows how data is retrieved from MongoDB collections to support SACCO operations such as managing members, loans, savings, and transactions. It demonstrates how the system is used to search, filter, and analyze data for reporting and decision-making

4. Retrieve all active members

```
db.Member.find({ status: "Active" }); Find loans for a specific member
```

5. Find loans for a specific member

```
db.Loan.find({ member_id: "Member-201" });
```

6. Get saving account balance

```
db.SavingAccount.find(
  { member_id: "Member-201" },
  { current_balance: 1, account_number: 1 }
);
```

7. Check overdue loan schedules

```
db.LoanSchedule.find({ status: "Overdue" });
```

8. Calculate total deposits per account

```
db.SavingsTransaction.aggregate([
  { $match: { transaction_type: "Deposit" } },
  { $group: { _id: "$account_id", totalDeposits: { $sum: "$amount" } } }
]);
```

The MongoDB implementation for Yisakal SACCO successfully models real-world SACCO operations using collections, embedded documents, and references. Schema validation ensures data integrity, indexing improves performance, and seed data provides realistic financial system simulation.

4.CONCLUSION

In conclusion, the Yisakal SACCO database project successfully designed and implemented an integrated database management system using both MySQL and MongoDB technologies. The system was developed to manage important SACCO operations such as member registration, savings accounts, loan processing, repayments, guarantor management, collateral tracking, fee management, and audit recording in an organized and efficient manner.

The relational database implemented in MySQL provides structured storage with proper tables, primary keys, foreign keys, and constraints to ensure data integrity, consistency, and reduced redundancy. The integrated database system that combines a normalized MySQL relational database and also a NoSQL database such as MongoDB which we have done them analogously or are corresponding to each other. Together, the two database systems improve scalability, performance, and flexibility of the SACCO management system.

Sample data insertion and SQL query execution were performed to test the functionality of the database. Different operations such as SELECT, UPDATE, ALTER, and DELETE queries demonstrated how the system can retrieve, modify, and manage data effectively. The implementation also showed that the database can support accurate reporting, secure record management, and efficient transaction processing.

Overall, the project achieved its objective of designing a reliable and scalable database system for Yisakal SACCO. The proposed system helps improve operational efficiency, data accuracy, decision-making, and financial service management within the organization. Future improvements may include integration with mobile banking systems, online services, advanced reporting tools, and enhanced security mechanisms.

5.REFERENCES

[1] *MySQL Workbench*, Oracle Corporation, Austin, TX, USA, 2026.

[2] *MongoDB Compass*, MongoDB, Inc., New York, NY, USA, 2026.

[3] *Lucidchart*, Lucid Software Inc., South Jordan, UT, USA, 2026.

[4] *draw.io*, JGraph Ltd., Northampton, UK, 2026.

[5] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[6] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.